

Complete Course & Laboratory Notes

PYTHON PROGRAMMING

5 Modules in Full · 12 Lab Questions Compiled & Captured ·
Concept-to-Code Tables · Checkpoint Questions · Viva Bank

COURSE CODE

1BPLC105B/205B

Credits	Teaching Hours (L : T : P : S)	Total Marks	CIE	SEE	Exam Hours	Examination Type
4	3 : 0 : 2 : 0 / week	100	50	50	3 Hours	Theory

by

Dr. Lokesh M R

Professor, Department of Information Science & Engineering
A J Institute of Engineering and Technology (AJIET), Mangaluru, Karnataka, India

Visvesvaraya Technological University (VTU), Belagavi · 2022/2025 Scheme · IPCC (Circular Ref.: VTU/BGM/IPCC 2025/3748, dated 24 Oct 2025)

First Edition · September 2026

PUBLICATION PARTICULARS**Course at a Glance**

Field	Details
Course	Python Programming (Integrated Theory & Practical Course — IPCC)
Course Code	1BPLC105B/205B
Semester	I / II
Credits	4
Teaching Hours/Week	3 : 0 : 2 : 0 (Lecture : Tutorial : Practical : Self-study)
Total Hours of Pedagogy	40 (theory) + 24 (practical)
Assessment	CIE 50 (theory 25 + practical 25) and SEE 50 · Total 100
SEE	3 hours, theory paper (see the assessment map inside)
Course Outcomes	CO1 - CO5 (verbatim reproduction in the Syllabus section)

What This Book Covers

Every topic line of the five modules is taught the way Python is actually written: the rule, a runnable fragment, and the exact console result that fragment produces. Every lab question — the eight fixed Part-A experiments (including their a/b sub-parts) and the four open-ended Part-B tools — is presented complete: approach, source, captured transcript, observations, result, viva trail and extension exercises.

- **Part A — Theory Modules 1 to 5:** one chapter per module, each with the module text verbatim, concept tables, code patterns and a worked demo program with captured output.
- **Part B — Laboratory Questions 1 to 12:** one chapter per question; questions 1-3 carry parts (a) and (b), and both programs are complete.
- **Part C — Appendices:** a Python quick reference, the IPCC record & rubric format with the qualification arithmetic, a consolidated viva bank, and the course references.

Provenance of the Code and Output**EVERYTHING PRINTED HERE WAS EXECUTED**

All twenty programs in this book (five module demos and fifteen lab solutions) were run on CPython 3.13 with NumPy 2.3 on the laboratory Linux machines in September 2026. Every transcript was captured from a real terminal session (a pseudo-terminal, so echoed input lands exactly where a student at the bench would see it); no output was typed by hand or edited afterwards. Where a program deliberately triggers a guard — division by zero, an assertion, a missing file — that run is printed too, because those transcripts are what the record is graded on.

Imprint

First Edition, September 2026 · Prepared for the VTU 2022/2025 scheme under the IPCC circular dated 24 Oct 2025 · Course materials follow the university-prescribed syllabus reproduced verbatim in the next-but-one section · Errors in printing, if any, are the author's; corrections are welcome through the department office.

FRONT MATTER

Preface

Python is the first programming course many of these students will finish with an opinion — and that is exactly what an integrated theory-and-lab scheme is for. The syllabus asks for both halves of the skill at once: reading a program critically (the modules) and producing one under observation (the lab). Books usually serve one half; this one serves both, in the same voice, with the same verified programs on both sides of every idea.

The rule this book keeps is simple: **no statement about output without the output**. Every code pattern shown in a module chapter was executed; every lab chapter carries the complete source, the captured session, and the analysis the examiner asks for orally. The three error types of Module 1 are not defined and abandoned — they are provoked on purpose later, in the assertion and exception transcripts of Lab Question 7, so that by the end of the course the student has seen a syntax error, a runtime error and a semantic error, and knows which one costs marks in the record.

Who This Book Is For

- **First-year students** taking 1BPLC105B/205B who must write the lab record, defend it in the internal assessment, and answer a 3-hour theory paper.
- **Laboratory instructors** running the 24 practical hours — each lab chapter is a complete bench session: program, runs (including guard runs), and viva questions with model answers.
- **Self-learners** who want the prescribed reading (How to Think Like a Computer Scientist, chapters 1–4, 5.1–5.4, 6.1–6.5, 7.1–7.8, 8.1–8.8, 9.1, 11.2–5.3, 12.1–12.2) walked at the exact depth the COs demand — no more, no less.

Conventions Used in This Book

Element	Meaning
Shaded box, “As prescribed (verbatim)”	The syllabus wording reproduced exactly — module text or lab question as issued.
Numbered \$x.y\$ sections	Every module chapter and every lab chapter uses the same section skeleton, so any record format can be filled by copying one column.
Code block with a dark title bar	Complete source file as executed; the filename in the bar is the file to create.
“Run n — ...” label + pale block	Captured terminal session: text the program printed and the input echoed at the prompt, from a real pty run.
Note / Warning boxes	The insight to write in the record / the mistake that loses marks in the viva.

— The Author

A J Institute of Engineering and Technology (AJIET), Mangaluru
September 2026

FRONT MATTER

Prescribed Syllabus — Verbatim

The course header, outcomes, all five modules, the practical components, resources and the IPCC assessment structure for Python Programming (1BPLC105B/205B) are reproduced below exactly as prescribed, so that every chapter of this book can be checked against the official wording.

Course Header

Field	Details
Course	PYTHON PROGRAMMING
Semester	I / II
Course Code	1BPLC105B/205B
CIE Marks	50
Teaching Hours/Week (L:T:P:S)	3 : 0 : 2 : 0
SEE Marks	50
Total Hours of Pedagogy (Theory and Lab hours)	40 + 24 (Practical)
Total Marks	100
Credits	4
Exam Hours	3
Examination type (SEE)	Theory

Course Outcome (Course Skill Set)

At the end of the course, the student will be able to:

CO1: Develop scripts using primitive language constructs of python.

CO2: Identify the methods to manipulate primitive python data structures.

CO3: Make use of Python standard libraries for programming.

CO4: Build scripts for performing file operations.

CO5: Illustrate the concepts of Object-Oriented Programming as used in Python.

Module-1

The way of the program: The Python programming language, what is a program? What is debugging? Syntax errors, Runtime errors, Semantic errors, Experimental debugging. Variables, Expressions and Statements: Values and data types, Variables, Variable names and keywords, Statements, Evaluating expressions, Operators and operands, Type converter functions, Order of operations, Operations on strings, Input, Composition, The modulus operator. Iteration: Assignment, Updating variables, the for loop, the while statement, The Collatz $3n + 1$ sequence, tables, two-dimensional tables, break statement, continue statement, paired data, Nested Loops for Nested Data. Functions: Functions with arguments and return values. Chapters: 1.1-1.7, 2.1-2.12, 3.3, 4.4, 4.5 · Number of Hours: 8

Module-2

Strings: Working with strings as single things, working with the parts of a string, Length, Traversal and the for loop, Slices, String comparison, Strings are immutable, the in and not in operators, A find function, Looping and counting, Optional parameters, The built-in find method, The split method, Cleaning up your strings, The string format method. Tuples: Tuples are used for grouping data, Tuple assignment, Tuples as return values, Composability of Data Structures. Lists: List values, accessing elements, List length, List membership, List operations, List slices, Lists are mutable, List deletion, Objects and references, Aliasing, cloning lists, Lists and for loops, List parameters, List methods, Pure functions and modifiers, Functions that produce lists, Strings and lists, list and range, Nested lists, Matrices. Chapter: 5.1, 5.2, 5.3 · Number of Hours: 8

Module-3

Dictionaries: Dictionary operations, dictionary methods, aliasing and copying. Numpy: About, Shape, Slicing, masking, Broadcasting, dtype. Files: About files, writing our first file, reading a file line-at-a-time, turning a file into a list of lines, Reading the whole file at once, working with binary files, Directories, fetching something from the Web. Chapter: 5.4, 6.1-6.5, 7.1-7.8 · Number of Hours: 8

Module-4

Modules: Random numbers, the time module, the math module, creating your own modules, Namespaces, Scope and lookup rules, Attributes and the dot Operator, Three import statement variants. Mutable versus immutable and aliasing. Object oriented programming: Classes and Objects — The Basics, Attributes, Adding methods to our class, Instances as arguments and parameters, Converting an instance to a string, Instances as return values. Chapter: 8.1-8.8, 9.1, 11.1 · Number of Hours: 8

Module-5

Object oriented programming: Objects are mutable, Sameness, Copying. Inheritance: Pure functions, Modifiers, Generalization, Operator Overloading, Polymorphism. Exceptions: Catching Exceptions, Raising your own exceptions. Chapter: 11.2.2-11.2.4, 11.3.2-11.3.9, 12.1, 12.2 · Number of Hours: 8

Practical Components of IPCC — Part A: Fixed Set of Experiments

1. **1 a.** Develop a python program to read 2 numbers from the keyboard and perform the basic arithmetic operations based on the choice. (1-Add, 2-Subtract, 3-Multiply, 4-Divide).
b. Develop a program to read the name and year of birth of a person. Display whether the person is a senior citizen or not.
2. **2 a.** Develop a program to generate Fibonacci sequence of length (N). Read N from the console.
b. Write a python program to create a list and perform the following operations: Inserting an element, Removing an element, Appending an element, Displaying the length of the list, Popping an element, Clearing the list.
3. **3 a.** Read N numbers from the console and create a list. Develop a program to print mean, variance and standard deviation with suitable messages.
b. Read a multi-digit number (as chars) from the console. Develop a program to print the frequency of each digit with a suitable message.
4. **4.** Develop a program to print 10 most frequently appearing words in a text file. [Hint: Use a dictionary with distinct words and their frequency of occurrences. Sort the dictionary in the reverse order of frequency and display the dictionary slice of the first 10 items.]
5. **5.** Develop a program to read 6 subject marks from the keyboard for a student. Generate a report that displays the marks from the highest to the lowest score attained by the student. [Read the marks into a 1-Dimensional array and sort using the Bubble Sort technique].
6. **6.** Develop a program to sort the contents of a text file and write the sorted contents into a separate text file. [Hint: Use string methods strip(), len(), list methods sort(), append(), and file methods open(), readlines(), and write()].
7. **7.** Develop a function named DivExp which takes TWO parameters a, b, and returns a value c ($c=a/b$). Write a suitable assertion for $a>0$ in the function DivExp and raise an exception for when $b=0$. Develop a suitable program that reads two console values and calls the function DivExp.
8. **8.** Define a function that takes TWO objects representing complex numbers and returns a new complex number with the sum of two complex numbers. Define a suitable class 'Complex' to represent the complex number. Develop a program to read N ($N \geq 2$) complex numbers and to compute the addition of N complex numbers.

Part B: Typical Open-Ended Experiments

1. **9.** Text Analysis Tool: Build a tool that analyses a paragraph: frequency of each word, longest word, number of sentences, etc.
2. **10.** Develop Data Summary Generator: Read a CSV file (like COVID data or weather stats), convert to dictionary form, and allow the user to run summary queries: max, min, average by column.

3. **11.** Develop Student Grade Tracker: Accept multiple students' names and marks. Store them in a list of tuples or dictionaries. Display summary reports (average, topper, etc.).
4. **12.** Develop a program to display contents of a folder recursively (Directory) having sub-folders and files (name and type).

Suggested Learning Resources

Text books:

1. Peter Wentworth, Jeffrey Elkner, Allen B. Downey and Chris Meyers — How to think like a computer scientist: learning with python 3. Green Tea Press, Wellesley, Massachusetts, 2020. <https://media.readthedocs.org/pdf/howtothink/latest/howtothink.pdf>

Reference books / Manuals:

1. Al Sweigart, "Automate the Boring Stuff with Python, 2nd Edition: Practical Programming for Total Beginners", 2nd Edition, No Starch Press, 2022. (Available under CC-BY-NC-SA license at <https://automatetheboringstuff.com/>)
2. Kyla McMullen, Elizabeth Matthews and Elizabeth Parsons — Programming with Python, Cengage, 2023.

Web links and Video Lectures (e-Resources):

- <https://www.learnbyexample.org/python/>
- <https://www.learnpython.org/>
- <https://pythontutor.com/visualize.html#mode=edit>

Teaching-Learning Process (Innovative Delivery Methods)

The following are sample strategies that educators may adopt to enhance the effectiveness of the teaching-learning process and facilitate the achievement of course outcomes:

1. Chalk and talk
2. PPT presentation
3. Demonstration
4. Problem-Based Learning (PBL)
5. Case-Based Teaching

Assessment Structure (IPCC)

CIRCULAR REF.: VTU/BGM/IPCC 2025/3748, DATED 24 OCT 2025

The assessment for each course is equally divided between Continuous Internal Evaluation (CIE) and the Semester End Examination (SEE), with each component carrying 50% weightage (i.e., 50 marks each). The CIE Theory component will be 25 marks and CIE Practical component will be 25 marks. The CIE Theory component consists of IA tests for 25 marks. The CIE Practical component for continuous assessments will be for 15 marks through rubrics and for lab Internal Assessment will be conducted for 10 marks through rubrics.

- To qualify and become eligible to appear for SEE, in the CIE theory component, a student must score at least 40% of 25 marks, i.e., 10 marks.
- To qualify and become eligible to appear for SEE, in the CIE Practical component, a student must secure a minimum of 40% of 25 marks, i.e., 10 marks.
- To pass the SEE, a student must secure a minimum of 35% of 50 marks, i.e., 18 marks.
- A student is deemed to have completed the course if the combined total of CIE and SEE is at least 40 out of 100 marks.

FRONT MATTER

Course Outcome → Chapter Mapping

Each chapter of this book declares the COs it serves. The map below is the quick reference; the verbatim module text at the head of every theory chapter is the authority.

Course Outcome	M1	M2	M3	M4	M5	Lab 1-3	Lab 4-8	Lab 9-12
CO1 — primitive constructs	●	○				●	○	○
CO2 — primitive data structures		●	●	○		●	●	○
CO3 — standard libraries	○	○	●	●	○		○	●
CO4 — file operations			●				●	●
CO5 — OOP concepts		○		●	●		●	○

● primary coverage · ○ supporting coverage. Lab numbering follows the syllabus: 1-3 include both a/b programs; 4-8 are the remaining fixed experiments; 9-12 are the open-ended tools.

Syllabus Compliance Map

Syllabus element (as printed)	Pages here	How this book discharges it
Module-1 (Way of the program · Variables · Iteration · Functions)	1-11	Theory chapter 1: three error types with provoked examples, expression evaluation, modulus, Collatz + 2-D tables, break/continue, nested loops, function arguments & returns. Demo m01.py captured.
Module-2 (Strings · Tuples · Lists)	6-9	Theory chapter 2: slicing, immutability, find/split/format, tuple assignment, list operations, aliasing vs clone, pure vs modifier, nested lists & matrices. Demo m02.py captured.
Module-3 (Dictionaries · NumPy · Files)	10-13	Theory chapter 3: dictionary methods and copying, ndarray shape/slice/mask/broadcast/dtype, all file-reading modes, binary files, directories, web fetch pattern. Demo m03.py captured.
Module-4 (Modules · OOP basics)	14-16	Theory chapter 4: import variants, namespace/scope lookup, attributes and dot, mutable defaults trap; classes, methods, instances as args/returns, <code>__str__</code> . Demo m04.py captured.
Module-5 (OOP contd · Inheritance · Exceptions)	17-21	Theory chapter 5: sameness & copying, inheritance, operator overloading, polymorphism, raising/catching. Demo m05.py captured.
Part-A experiments 1-8 (with a/b parts)	22-45	Fifteen complete programs (1a...8), each with captured transcripts including guard runs (divide-by-zero, invalid choice, negative year, empty input, b=0, N<2).
Part-B open-ended 9-12	46-57	Four tool chapters: design decisions argued, test plan with runs, rubric alignment.
Assessment structure (CIE 25+25 / SEE 50, eligibility)	Appx B	Rubric tables, qualification arithmetic worked, 3-hour answer strategy.

READING THE PAGE NUMBERS ABOVE

Theory chapters are numbered **Module 1-5**; laboratory chapters are numbered **Experiment 1-12** exactly as the syllabus numbers the questions. A pair like “1a/1b” is one Experiment chapter containing both programs, since the question is one entry in the official list.

Assessment Quick View

Component	Marks	What carries it
CIE Theory	25	Two IA tests on the module content (theory syllabus)
CIE Practical — continuous	15	Lab record evaluated through rubrics (every 2 experiments)
CIE Practical — internal	10	Lab internal assessment through rubrics (end-semester, at the bench)
SEE (theory)	50	3-hour theory paper covering all five modules

Eligibility for SEE: $\geq 10/25$ in CIE Theory and $\geq 10/25$ in CIE Practical. Pass in SEE: $\geq 18/50$ (35%). Course completion: CIE + SEE $\geq 40/100$. Worked examples in Appendix B.

FRONT MATTER**List of Contents**

Page numbers in roman (i–xi) are front matter; the body restarts at arabic 1 and runs continuously through the appendices — both match the printed footers and the PDF bookmarks.

Title Page	i
Course at a Glance	ii
Preface	iii
Prescribed Syllabus — Verbatim	iv
CO & Compliance Maps	vii
List of Contents	ix
PART A — Theory Modules 1-5	1
Module 1 — The Way of the Program, Variables, Iteration & Functions	2
1.1 Concepts in Focus	2
1.2 Worked Demo Program	3
1.3 Execution (Captured)	4
1.4 Quick-Reference Summary	4
1.5 Checkpoint Questions	4
1.6 Exercises & Practice	5
Module 2 — Strings, Tuples and Lists	6
2.1 Concepts in Focus	6
2.2 Worked Demo Program	7
2.3 Execution (Captured)	7
2.4 Quick-Reference Summary	8
2.5 Checkpoint Questions	8
2.6 Exercises & Practice	9
Module 3 — Dictionaries, NumPy and File Handling	10
3.1 Concepts in Focus	10
3.2 Worked Demo Program	11
3.3 Execution (Captured)	11
3.4 Quick-Reference Summary	12
3.5 Checkpoint Questions	12
3.6 Exercises & Practice	12
Module 4 — Modules, Namespaces and the Basics of Classes	14
4.1 Concepts in Focus	14
4.2 Worked Demo Program	15
4.3 Execution (Captured)	15
4.4 Quick-Reference Summary	15
4.5 Checkpoint Questions	16
4.6 Exercises & Practice	16
Module 5 — Objects, Inheritance, Operator Overloading and Exceptions	17
5.1 Concepts in Focus	17
5.2 Worked Demo Program	18
5.3 Execution (Captured)	18
5.4 Quick-Reference Summary	19
5.5 Checkpoint Questions	19
5.6 Exercises & Practice	20
PART B — Laboratory Questions 1-12	21
Experiment 1 — Calculator by Choice, and a Senior-Citizen Test	22
1.1 Approach	22

1.2 Program Code	22
1.3 Execution & Transcripts	23
1.4 Observations & Analysis	24
1.5 Result	24
1.6 Viva Voce Questions	24
1.7 Exercises & Extensions	25
Experiment 2 — Fibonacci to Length N, and the Full List Lifecycle	26
2.1 Approach	26
2.2 Program Code	26
2.3 Execution & Transcripts	27
2.4 Observations & Analysis	27
2.5 Result	27
2.6 Viva Voce Questions	28
2.7 Exercises & Extensions	28
Experiment 3 — Mean, Variance, Standard Deviation — and Digit Frequencies	29
3.1 Approach	29
3.2 Program Code	29
3.3 Execution & Transcripts	30
3.4 Observations & Analysis	31
3.5 Result	31
3.6 Viva Voce Questions	31
3.7 Exercises & Extensions	31
Experiment 4 — Ten Most Frequent Words in a Text File	32
4.1 Approach	32
4.2 Program Code	32
4.3 Execution & Transcripts	33
4.4 Observations & Analysis	33
4.5 Result	33
4.6 Viva Voce Questions	33
4.7 Exercises & Extensions	34
Experiment 5 — Marks Report by Bubble Sort (Highest → Lowest)	35
5.1 Approach	35
5.2 Program Code	35
5.3 Execution & Transcripts	36
5.4 Observations & Analysis	36
5.5 Result	36
5.6 Viva Voce Questions	36
5.7 Exercises & Extensions	37
Experiment 6 — Sort a File's Lines into a Separate Sorted File	38
6.1 Approach	38
6.2 Program Code	38
6.3 Execution & Transcripts	39
6.4 Observations & Analysis	39
6.5 Result	39
6.6 Viva Voce Questions	39
6.7 Exercises & Extensions	39
Experiment 7 — DivExp with an Assertion and a Raised Exception	40
7.1 Approach	40
7.2 Program Code	40
7.3 Execution & Transcripts	40
7.4 Observations & Analysis	41
7.5 Result	41
7.6 Viva Voce Questions	41

7.7 Exercises & Extensions	41
Experiment 8 — Complex Numbers as Objects — Summing N with a Pure Adder	43
8.1 Approach	43
8.2 Program Code	43
8.3 Execution & Transcripts	44
8.4 Observations & Analysis	44
8.5 Result	44
8.6 Viva Voce Questions	44
8.7 Exercises & Extensions	45
Experiment 9 — Text Analysis Tool (Open-Ended)	46
9.1 Approach	46
9.2 Program Code	46
9.3 Execution & Transcripts	47
9.4 Observations & Analysis	47
9.5 Result	47
9.6 Viva Voce Questions	47
9.7 Exercises & Extensions	48
Experiment 10 — CSV Data Summary Generator (Open-Ended)	49
10.1 Approach	49
10.2 Program Code	49
10.3 Execution & Transcripts	50
10.4 Observations & Analysis	50
10.5 Result	50
10.6 Viva Voce Questions	50
10.7 Exercises & Extensions	51
Experiment 11 — Student Grade Tracker (Open-Ended)	52
11.1 Approach	52
11.2 Program Code	52
11.3 Execution & Transcripts	53
11.4 Observations & Analysis	53
11.5 Result	53
11.6 Viva Voce Questions	53
11.7 Exercises & Extensions	54
Experiment 12 — Recursive Folder Listing (Open-Ended)	55
12.1 Approach	55
12.2 Program Code	55
12.3 Execution & Transcripts	56
12.4 Observations & Analysis	56
12.5 Result	56
12.6 Viva Voce Questions	56
12.7 Exercises & Extensions	57
PART C — Appendices	58
Appendix A — Python Quick Reference	59
Appendix B — Record Format & Assessment Rubrics	62
Appendix C — Viva Voce Bank	64
Appendix D — References & Further Reading	66
About the Author	66

P A R T



Theory — Modules 1 to 5

Every syllabus line of the five modules, taught through code that was actually run: each chapter carries the module text verbatim, the concept map with runnable fragments, one worked demo program, its captured session, a quick-reference summary keyed to the prescribed wording, and checkpoint questions in IA-paper shape.

IN THIS PART

- Module 1 — errors, variables, expressions, loops, functions
- Module 2 — string methods, tuple assignment, list operations
- Module 3 — dict ops, ndarray basics, reading & writing files
- Module 4 — import variants, scope rules, classes & instances
- Module 5 — sameness & copying, inheritance, overloading, exceptions

Module 1 — The Way of the Program, Variables, Iteration & Functions

Module 1 is where the course earns its “integrated” label: the way of the program, the three error kinds, the arithmetic of expressions, the two loops, and the first functions — every idea introduced the way it will be examined, with the code that proves it.

COURSE OUTCOMES	CO1 (primary) · CO5 groundwork
CONTACT & LAB LINK	8 lecture hours · supports Lab Experiments 1–3
PRESCRIBED READING	How to Think Like a Computer Scientist: 1.1–1.7, 2.1–2.12, 3.3, 4.4, 4.5

AS PRESCRIBED (VERBATIM)

The way of the program: The Python programming language, what is a program? What is debugging? Syntax errors, Runtime errors, Semantic errors, Experimental debugging. Variables, Expressions and Statements: Values and data types, Variables, Variable names and keywords, Statements, Evaluating expressions, Operators and operands, Type converter functions, Order of operations, Operations on strings, Input, Composition, The modulus operator. Iteration: Assignment, Updating variables, the for loop, the while statement, The Collatz $3n + 1$ sequence, tables, two-dimensional tables, break statement, continue statement, paired data, Nested Loops for Nested Data. Functions: Functions with arguments and return values. Chapters: 1.1–1.7, 2.1–2.12, 3.3, 4.4, 4.5 · Number of Hours: 8

1.1 Concepts in Focus

A **program** is a sequence of instructions that tells the computer what to do; the sequence is exact, and the three ways it can be wrong are the three error kinds the syllabus names. Keep one example of each in the record — the viva asks for them constantly:

Error kind	Example	What happens
Syntax error	<code>print("hi"</code>	Parser refuses the whole file; nothing runs until it is fixed. The traceback points at a line — often one before the real mistake.
Runtime error	<code>10/0</code>	Program starts, then dies mid-run with a named exception (<code>ZeroDivisionError</code>). Also called an exception.
Semantic error	<code>mean computed as total*n</code>	Runs cleanly, prints the wrong thing. No traceback — you are the debugger; this is the one that costs SEE marks silently.

Experimental debugging is the prescribed method: run small experiments that separate the hypotheses, change one thing at a time, keep the smallest failing example.

Values, variables, expressions

- Basic values: `5` (int), `2.5` (float), `"py"` (str), `True` (bool). `type(x)` reports the class of the value.
- A **variable** is a name referring to a value: `x = 5` is an assignment statement — the `=` is “becomes equal to”, not algebra. Names must start with a letter/underscore, are case-sensitive, and may not be one of the ~35 **keywords** (`if`, `for`, `while`, `def`, `return`, `class`...; `help("keywords")` lists them).
- **Evaluating expressions**: operators work on operands following precedence — power → unary minus → `*` / `//` `%` → `+` `-`; parentheses override. Same-precedence chains run left to right, except `**`, which runs right to left (`2**3**2 == 512`).
- **Type converters**: `int("42")`, `float("4.2")`, `str(42)`, `bool(1)`. `int("4.2")` raises a `ValueError` — convert via `float`.
- **Strings + operators**: `+` concatenates, `*` repeats (both need at least one string operand); `input()` always yields a string — feeding it to arithmetic without `int()/float()` is the course’s most common semantic error.

- **The modulus operator** `%` returns the remainder: `17 % 5 == 2`, and it pairs with floor division `//`: `a == (a//b)*b + a%b`. Divisibility tests (`n % 2 == 0`) and digit peeling (`n % 10`) are its two lab faces.

Iteration

- The **for loop** walks a sequence: `for i in range(5):` gives 0..4. The **while statement** tests a condition each pass — stop must happen inside, or the loop never ends.
- **Updating variables**: `total += n` (accumulator), `count = count + 1` (tally). Updating before initialising is a runtime error (`NameError`).
- **Collatz 3n+1**: halve an even term; triple-and-add-one an odd term; the sequence is conjectured to reach 1 for every start. The canonical while-loop exercise — see the demo below.
- **Tables and two-dimensional tables**: nested `for`s, outer rows / inner columns, aligned with `print(f"{v:4d}", end="")`.
- **break** leaves the innermost loop at once; **continue** skips to the next iteration; both apply to `for` and `while`.
- **Paired data & nested loops for nested data**: `for a, b in pairs:` unpacks tuples per iteration; when the data itself is a list of lists, loop the container structure — one loop level per data level.

Functions

`def name(parameters):` then a body; `return value` hands a result back. Arguments bind to parameters in order (or by name at call time). A call composes into any expression. No `return` means the function returns `None`. Functions are the unit the lab report (“Develop a function named `DivExp...`”) grades.

1.2 Worked Demo Program

The demo exercises every headline of the module in one sitting: type conversion of `input()`, the modulus/floor pair, a Collatz function with a return value, a table of Collatz lengths (iteration + f-string layout), and an accumulator loop (updating variables).

Run from the working directory, exactly as executed for this book:

```
python3 m01.py
```

m01.py · complete source (30 lines, max width 71 cols)

```
# Module 1 demo -- program errors, variables, iteration, functions
# Run   : python3 m01.py

def collatz(n):
    """Steps for the 3n+1 sequence started at n to reach 1."""
    steps = 0
    while n != 1:
        n = n // 2 if n % 2 == 0 else 3 * n + 1
        steps += 1
    return steps

value = input("Enter an integer: ")      # input() always yields a str
number = int(value)                     # type converter: str -> int
print("As int:", number, "| as float:", float(number))

print()
print("Modulus & floor division:")
print("17 // 5 =", 17 // 5, "  17 % 5 =", 17 % 5)

print()
print("Collatz table (start, steps to reach 1):")
for start in range(3, 8):
    print(f"{start:>2d} -> {collatz(start):>2d} steps")

print()
print("Updating variables with a for loop:")
total = 0
for k in range(1, number + 1):
```

```
total = total + k          # also writable as total += k
print("1 + ... +", number, "=", total)
```

1.3 Execution (Captured)

Captured terminal session — the program's lines and the echoed input as typed at the bench:

The captured session — input 12

```
Enter an integer: 12
As int: 12 | as float: 12.0

Modulus & floor division:
17 // 5 = 3   17 % 5 = 2

Collatz table (start, steps to reach 1):
3 -> 7 steps
4 -> 2 steps
5 -> 5 steps
6 -> 8 steps
7 -> 16 steps

Updating variables with a for loop:
1 + ... + 12 = 78
```

1.4 Quick-Reference Summary

Syllabus phrase	The one-line Python
what is debugging?	finding and removing errors: syntax → read the traceback; runtime → read the exception name; semantic → test hypotheses in small runs
Type converter functions	<code>int(s)</code> · <code>float(s)</code> · <code>str(v)</code>
Order of operations	<code>()</code> → <code>**</code> (right→left) → unary <code>-</code> → <code>*</code> <code>//</code> <code>%</code> → <code>+</code> <code>-</code>
The modulus operator	<code>a % b</code> ; with <code>//</code> : <code>a == (a//b)*b + a%b</code>
the for loop / the while statement	<code>for x in seq:</code> · <code>while cond:</code> — finite because the sequence ends / because the update stops it
tables, two-dimensional tables	nested for over rows×columns with <code>end=""</code> and width formatting
break / continue	exit innermost loop / skip one iteration
Functions with arguments and return values	<code>def f(a, b): ... return v</code>

1.5 Checkpoint Questions

- What is the difference between a syntax error and a semantic error, and which one produces a traceback?
- Why does `input()` usually need a type converter before arithmetic? Show both directions of the mistake.
- Evaluate: `17 // 5`, `17 % 5`, `2 ** 3 ** 2`. Justify each with the precedence rules.
- Write the Collatz step function in two lines and state what `collatz(7)` returns after computing by hand (16).
- When does `break` differ from `continue`? Give a loop where each is the correct choice.
- Why must a while-loop body change a variable tested in the condition? Name the failure mode if it does not.

Model answers to the most-asked questions

Q1. What is the difference between a syntax error and a semantic error, and which one produces a traceback?

Ans. A syntax error breaks parsing before any instruction runs — the interpreter prints a traceback and

refuses to start. A semantic error means the program runs but does the wrong thing; there is no trace-back at all, so it is found by testing, not by reading. Runtime errors (exceptions) sit between: parsing succeeded, execution began, then something failed at a named point.

Q4. Write the Collatz step function in two lines and state what `collatz(7)` returns after computing by hand (16).

Ans. `collatz(7)`: $7 \rightarrow 22 \rightarrow 11 \rightarrow 34 \rightarrow 17 \rightarrow 52 \rightarrow 26 \rightarrow 13 \rightarrow 40 \rightarrow 20 \rightarrow 10 \rightarrow 5 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$ is 16 steps — matching the table row in the demo output, which the program computed the same way.

1.6 Exercises & Practice

1. Predict, then run: `print(1 + 2 * 3 ** 2)` and `print(10 % 3 + 10 // 3)`.
2. Print a two-dimensional multiplication table 1..9 using nested for loops and `f"{i*j:4d}"`.
3. Loop with a break: print numbers until the first multiple of 7 above 50. Then redo it with a while and no break, comparing clarity.
4. Write `digit_count(n)` with modulus and floor division only (no `str()`), and verify against `len(str(n))` for 12345.

Module 2 — Strings, Tuples and Lists

Module 2 turns the primitive value types into structures you can cut, count and compare. Strings and lists share the same navigation grammar — index, traverse, slice — but opposite constitutions: one cannot change, the other can; tuples group without mutating.

COURSE OUTCOMES	CO2 (primary) · CO1 groundwork
CONTACT & LAB LINK	8 lecture hours · supports Lab Experiments 2b, 3a, 3b
PRESCRIBED READING	How to Think Like a Computer Scientist: 5.1, 5.2, 5.3 (with 2.x string basics)

AS PRESCRIBED (VERBATIM)

Strings: Working with strings as single things, working with the parts of a string, Length, Traversal and the for loop, Slices, String comparison, Strings are immutable, the in and not in operators, A find function, Looping and counting, Optional parameters, The built-in find method, The split method, Cleaning up your strings, The string format method. Tuples: Tuples are used for grouping data, Tuple assignment, Tuples as return values, Composability of Data Structures. Lists: List values, accessing elements, List length, List membership, List operations, List slices, Lists are mutable, List deletion, Objects and references, Aliasing, cloning lists, Lists and for loops, List parameters, List methods, Pure functions and modifiers, Functions that produce lists, Strings and lists, list and range, Nested lists, Matrices. Chapter: 5.1, 5.2, 5.3 · Number of Hours: 8

2.1 Concepts in Focus

Strings

- **As single things vs parts:** a string is one value (`"banana"`) and simultaneously an ordered sequence of characters, indexed `0..len(s)-1` or `-len(s)..-1`.
- **Length & traversal:** `len(s)`; `for ch in s:` walks characters — the loop is the traversal, no index bookkeeping needed. Indexing by hand needs `range(len(s))`.
- **Slices** `s[i:j]` take `i` up-to (not including) `j`; either end may be omitted; `s[::-1]` reverses. A slice of a string is a new string — never an alias.
- **Comparison** is lexicographic with Unicode code points: `"apple" < "banana"`, but `"Zoo" < "apple"` (capitals sort first) — normalise with `.lower()` before sorting text.
- **Immutable:** `s[0] = "J"` raises `TypeError`. To “change” a string, build a new one: `s = "J" + s[1:]`.
- **in / not in** test substring membership in one boolean expression.
- **find as function, then as method:** write `find(word, ch, start=0)` with an optional parameter — then see the built-in `word.find(ch, start)`, which returns `-1` when absent. Looping and counting over it (`pos = s.find(ch, pos+1)`) is the same pattern as the syllabus’s hand-written find.
- **split / cleaning / format:** `s.split()` breaks on whitespace (or a given delimiter) into a list; `strip()` removes surrounding characters (punctuation, whitespace); `"{} + {}".format(a, b)` (or f-strings) build display strings — every record-friendly line in this book uses them.

Tuples

A tuple groups fixed parts: `t = (2026, "Sep")`. **Tuple assignment** packs or unpacks in one statement — `a, b = b, a` swaps without a temp; functions **return tuples** to hand back several values. Tuples are **composable**: list of tuples, tuple of tuples, tuple as dict key — the pairing device used again in Module 3.

Lists

- Same navigation as strings — index, `len`, `in`, slices — but lists are **mutable**: `nums[1] = 9` is legal; `del nums[0]`, `nums.append(x)`, `nums.pop(i)`, `nums.clear()`, `nums.sort()`, `nums.reverse()` are the lab-working six.

- **List operations:** `+` concatenates, `*` repeats, and slicing `nums[1:3]` returns a new list (a copy of that window).
- **Objects and references / aliasing:** assignment copies the reference, not the data — `b = a` makes both names one list, so `b.append(1)` shows in `a`. Clone to break the tie: `b = a[:]` or `b = list(a)`. (NumPy arrays and strings behave by the same rule; strings just cannot mutate.)
- **List parameters:** a function receives the reference. A **modifier** mutates the caller's list (`def add_end(l): l.append(9)`); a **pure function** returns new data and leaves the argument alone. Choosing one and saying so is part of the record.
- **Producers:** `list(range(5))` → `[0,1,2,3,4]`; `list("abc")` → `['a','b','c']`; `split()` → words. **Nested lists** are rows of a **matrix**: `m[i][j]`, and comprehension `[[r*c for c in range(3)] for r in range(3)]` builds a table — the two-dimensional table from Module 1, now as data.

2.2 Worked Demo Program

The demo walks one full lap of the module: clean a string, split it, slice and reverse it, prove immutability with a caught `TypeError`, pack/unpack a tuple, mutate a list, pass a slice as parameter, and sum a nested-list diagonal.

Run from the working directory, exactly as executed for this book:

```
python3 m02.py
```

m02.py · complete source (34 lines, max width 79 cols)

```
# Module 2 demo -- strings, tuples, lists
# Run : python3 m02.py

word = " Python-Lab-2026 "
clean = word.strip()
print("strip() ->", repr(clean))
parts = clean.split("-")
print("split('-') ->", parts)
print("format -> {} has {} parts".format(clean, len(parts)))
print("slice/upper ->", clean[:6].upper(), "| reversed:", clean[::-1])
print("'Py' in clean:", "Py" in clean, '| "x" not in clean:', "x" not in clean)

try:
    clean[0] = "J" # strings are immutable
except TypeError as e:
    print("immutable ->", e)

point = (10, 20)
x, y = point # tuple assignment (unpacking)
print("\ntuple unpack:", x, y, "| swap works:", (y, x) == (20, 10))

nums = [4, 8, 15, 16, 23, 42]
nums[1] = 9 # lists are mutable
nums.append(99)
print("list mutable :", nums)
print("len:", len(nums), "| 15 in nums:", 15 in nums)

def first_two(items): # list as parameter, no copy
    return items[:2]
print("slice param :", first_two(nums))

matrix = [[1, 2], [3, 4], [5, 6]] # nested list = table
trace = sum(matrix[i][i] for i in range(2))
print("nested sum :", trace, "(diagonal 1 + 4)")
```

2.3 Execution (Captured)

Captured terminal session — the program's lines and the echoed input as typed at the bench:

Captured session

```

strip()      -> 'Python-Lab-2026'
split('-')   -> ['Python', 'Lab', '2026']
format       -> Python-Lab-2026 has 3 parts
slice/upper  -> PYTHON | reversed: 6202-baL-nohtyP
"Py" in clean: True | "x" not in clean: True
immutable    -> 'str' object does not support item assignment

tuple unpack: 10 20 | swap works: True
list mutable : [4, 9, 15, 16, 23, 42, 99]
len: 7 | 15 in nums: True
slice param  : [4, 9]
nested sum   : 5 (diagonal 1 + 4)

```

2.4 Quick-Reference Summary

Syllabus phrase	Python in one line
Traversal and the for loop	<code>for ch in s:</code>
Slices	<code>s[1:4]</code> , <code>s[:2]</code> , <code>s[::-1]</code> (new string)
Strings are immutable	<code>s[0]="J"</code> → <code>TypeError</code>
Optional parameters	<code>def find(s, ch, start=0):</code>
The built-in find / split	<code>s.find("an", 3)</code> → index or <code>-1</code> · <code>s.split("-")</code> → list
Cleaning up / format	<code>s.strip(punct).lower()</code> · <code>"{}->{}".format(a, b)</code>
Tuple assignment / as return	<code>a, b = b, a</code> · <code>return (q, r)</code>
Aliasing vs cloning lists	<code>b = a</code> (one list) · <code>b = a[:]</code> (two lists)
Pure functions and modifiers	returns new data vs mutates in place
list and range	<code>list(range(2, 10, 3))</code> → <code>[2, 5, 8]</code>
Nested lists, matrices	<code>m[row][col]</code> ; build rows with a comprehension

2.5 Checkpoint Questions

1. Give the two reasons a slice never creates an alias, but `b = a` does.
2. What does `"Python"[:3] + "Python"[3:] == "Python"` evaluate to, and which string property does it celebrate?
3. Write `count_char(s, ch)` twice: with a hand-written find loop, and with `s.count(ch)`. Explain the `-1` convention.
4. Why does `nums.sort()` return `None`? What class of mistake is `x = nums.sort()`?
5. Explain tuple assignment as the mechanism behind `a, b = b, a`, and why a function would return a tuple.
6. How is a 3×3 matrix represented as a nested list? Write the expression for its trace.

Model answers to the most-asked questions

Q2. What does `"Python"[:3] + "Python"[3:] == "Python"` evaluate to, and which string property does it celebrate?

Ans. True (value-equal). Slicing splits the string at any index and re-concatenation reproduces it — the slice returns new strings, so nothing mutated the original, which is exactly what immutability guarantees: every operation that looks like “taking part” is really building a new whole.

Q4. Why does `nums.sort()` return `None`? What class of mistake is `x = nums.sort()`?

Ans. Because `sort()` is a modifier — it reorders the list in place and returns `None`, a deliberate signal that no new list was produced. Assigning `x = nums.sort()` stores `None`; the classic semantic error of the module, and the reason pure-function style (`sorted(nums)`) exists side by side.

2.6 Exercises & Practice

1. Input a full name; print initials, surname length, and the name in “SURNAME, Given” format using slices and format.
2. Write `is_palindrome(s)` with the reversed-slice comparison; test on 'noon', 'Banana'.
3. Given `marks = [45, 90, 12, 77]`: produce sorted-copy, reversed view, and sum of first two — then the same with only list methods. Which is pure, which modifies?
4. Build a 4×4 times table with a nested comprehension, then print the anti-diagonal sum.

Module 3 — Dictionaries, NumPy and File Handling

Module 3 supplies the two lookup powers of the course — keys instead of positions (dictionaries), vectors instead of loops (NumPy) — and then opens the outside world: files read, written, streamed, and fetched.

COURSE COMES	CO3 + CO4 (primary) · CO2 supporting
CONTACT & LAB LINK	8 lecture hours · supports Lab Experiments 4, 6, 9, 10, 12
PRESCRIBED READING	How to Think Like a Computer Scientist: 5.4 (dicts), 6.1–6.5 (files/web); NumPy sections 7.1–7.8

AS PRESCRIBED (VERBATIM)

Dictionaries: Dictionary operations, dictionary methods, aliasing and copying. Numpy: About, Shape, Slicing, masking, Broadcasting, dtype. Files: About files, writing our first file, reading a file line-at-a-time, turning a file into a list of lines, Reading the whole file at once, working with binary files, Directories, fetching something from the Web. Chapter: 5.4, 6.1–6.5, 7.1–7.8 · Number of Hours: 8

3.1 Concepts in Focus

Dictionaries

- **Key→value mapping**, any hashable key; created as `d = {}` or `dict(name=..., year=2020)`. **Operations:** `d[k] = v`, `d[k]` (KeyError if absent), `del d[k]`, `k in d`, `len(d)`.
- **Methods:** `get(k, default)` (the graceful lookup), `keys()` / `values()` / `items()` for traversal, `pop(k)`, `update()`, `clear()`.
- **Aliasing and copying:** `b = a` shares storage; `b = a.copy()` is a top-level copy — nested values are still shared (the same distinction the lab rubric probes in Experiment 8's fold over dicts). The classic counting idiom: `freq[w] = freq.get(w, 0) + 1` — one line that powers Labs 3b, 4 and 9.
- Idiom for word-frequency sorting: `sorted(d.items(), key=lambda kv: -kv[1])` — the dictionary ordered by value, then sliced to the first 10 (Lab 4 verbatim).

NumPy

- **About:** the array module — one dtype per array, dense memory, whole-array arithmetic. `np.array([[1,2,3],[4,5,6]])`.
- **Shape** (rows, cols); **dtype** is reported by `arr.dtype` (int64/float64) — the difference between `arr/2` meaning halves and lists meaning truncation.
- **Slicing:** `arr[:, 1]` a column, `arr[0, :2]` row prefix; unlike Python lists, array slices are views — changing them changes the original (aliasing again, by design).
- **Masking:** boolean arrays select — `arr[arr > 3]` is the filter that replaces a hand loop; `arr[(arr % 2) == 0]` keeps the evens.
- **Broadcasting:** shapes stretch to meet — adding a length-3 row vector to a 2×3 matrix adds it to every row; a scalar reaches everywhere. It is why NumPy replaces most of the nested loops written with lists.

Files

- **Opening:** `f = open("name.txt", mode)` with modes `r` (read, default), `w` (truncate!), `a` (append), `r+`, and `b` added for binary. Always close — `with open(...) as f:` closes automatically and is the style of this book.
- **Writing:** `f.write("line\n")` — you add the newline.
- **Reading** three ways: `readline()` /for-loop line-at-a-time (streaming, memory-safe), `readlines()` (list of lines — exactly the lab 6 hint), `read()` (whole file as one string).
- **Binary:** open `"rb"`; bytes arrive — `f.read(11)` yields `b'first line\n'`-style literals; needed for images, audio, and any exact-byte job.

- **Directories:** `os.listdir(path)` lists, `os.path.join` assembles, `os.path.isdir/exists/getsize` inspect, `os.makedirs` creates; `os.walk` is the recursive engine (Lab 12).
- **Fetching something from the Web:** `from urllib.request import urlopen; urlopen(url).read().decode()` returns page text — the same open/read/close story, with a network as the pipe.

3.2 Worked Demo Program

The demo counts letters into a dictionary, shows get-default behaviour, proves the aliasing/copying split on the same dict, runs a 2×3 ndarray through shape/dtype/slice/mask/ broadcast, then writes a three-line file, reads it back as a list of lines and as raw bytes, and deletes it.

Run from the working directory, exactly as executed for this book:

```
python3 m03.py
```

m03.py · complete source (34 lines, max width 79 cols)

```
# Module 3 demo -- dictionaries, numpy basics, files & directories
# Run   : python3 m03.py
import os
import numpy as np

freq = {}
for letter in "banana":
    freq[letter] = freq.get(letter, 0) + 1
print("dict from loop   :", freq)
print("get() with default:", freq.get("a", 0), "|", freq.get("z", 0))
shared = freq
copied = freq.copy()
shared["z"] = 9
print("alias sees write :", freq)
copied["z"] = 0
print("copy then fix    :", freq["z"], "-> independent now")

arr = np.array([[1, 2, 3], [4, 5, 6]])
print("\nndarray:\n", arr)
print("shape:", arr.shape, "| dtype:", arr.dtype)
print("column slice:", arr[:, 1], "| mask > 3:", arr[arr > 3])
print("broadcast add:", arr + np.array([10, 20, 30]))

fname = "demo_notes.txt"
with open(fname, "w") as f:
    f.write("first line\nsecond line\nthird line\n")
with open(fname) as f:
    lines = [ln.strip() for ln in f.readlines()]
print("\nfile as list of lines:", lines)
with open(fname, "rb") as f:
    raw = f.read()
print("binary read:", len(raw), "bytes | head:", raw[:11])
print("exists:", os.path.exists(fname), "| size:", os.path.getsize(fname), "B")
os.remove(fname)
```

3.3 Execution (Captured)

Captured terminal session — the program's lines and the echoed input as typed at the bench:

Captured session

```
dict from loop   : {'b': 1, 'a': 3, 'n': 2}
get() with default: 3 | 0
alias sees write : {'b': 1, 'a': 3, 'n': 2, 'z': 9}
copy then fix    : 9 -> independent now

ndarray:
[[1 2 3]
 [4 5 6]]
```

```

shape: (2, 3) | dtype: int64
column slice: [2 5] | mask > 3: [4 5 6]
broadcast add: [[11 22 33]
                [14 25 36]]

file as list of lines: ['first line', 'second line', 'third line']
binary read: 34 bytes | head: b'first line\n'
exists: True | size: 34 B

```

3.4 Quick-Reference Summary

Syllabus phrase	Python in one line
Dictionary operations	<code>d[k]=v · del d[k] · k in d · len(d)</code>
dictionary methods	<code>get · keys · values · items · pop · update</code>
dict aliasing vs copying	<code>b=a</code> shares · <code>b=a.copy()</code> separates top level
Shape / dtype	<code>arr.shape == (2, 3) · arr.dtype == int64</code>
Slicing / masking	<code>arr[:, 1]</code> (a view) · <code>arr[arr>3]</code>
Broadcasting	<code>arr + np.array([10, 20, 30])</code> → every row gains
writing / reading modes	<code>with open(p, "w") as f: f.write(s) · readlines() · read() · read(n)</code>
binary files	<code>open(p, "rb").read()</code> → bytes
Directories / web	<code>os.listdir, os.walk, os.path.getsize · urlopen(url).read().decode()</code>

3.5 Checkpoint Questions

1. Why is a dictionary the right structure for word counting, and where does a list fail at the same job?
2. Write the three-line counting loop without `get()`, then with it; explain which line raises `KeyError` on which input.
3. Contrast NumPy column slicing with list slicing: which returns data you can edit without touching the original?
4. What does broadcasting do when a (3,) array is added to a (2,3) array? Give the result for `[1 2 3]/[4 5 6] + [10 20 30]`.
5. Compare `read()`, `readlines()` and a for-loop over the file object: memory, return type, and which one the Lab-6 hint demands.
6. Explain the difference between `w` and `a` modes, and why `with` makes the record's file code safer.

Model answers to the most-asked questions

Q3. Contrast NumPy column slicing with list slicing: which returns data you can edit without touching the original?

Ans. A NumPy slice `arr[:, 1]` is a view — writing through it edits the original array, because the slice shares memory; a Python list slice always builds a copy. One of these is a feature (in-place column fixes), the other a safety (no accidental alias). The viva loves the contrast.

Q5. Compare `read()`, `readlines()` and a for-loop over the file object: memory, return type, and which one the Lab-6 hint demands.

Ans. `read()` returns one str of the whole file (two copies of its size in memory worst case); `readlines()` returns a list of str — one element per line, which is exactly what Lab 6 must sort; iterating the file object yields lines one at a time with constant memory — the streaming choice for big files.

3.6 Exercises & Practice

1. Count vowels in a paragraph into a dict; print sorted by frequency descending.
2. Read a CSV of your choice with one pass, building dict per row (zip header with row); print column sums — without NumPy, then with `arr.sum(axis=0)`.

3. Write ten lines to a file, read them back reversed with `readlines()[::-1]`, and save as `rev.txt`; report byte counts of both.
4. Fetch `https://example.com` with `urlopen`, print its length and the `<title>` line; then rerun with the URL misspelled and read the exception.

Module 4 — Modules, Namespaces and the Basics of Classes

Module 4 moves the student from scripts to systems: reusable code behind imports, names that obey lookup rules, and the first class — data and behaviour in one object, the foundation Labs 7 and 8 grade.

COURSE OUTCOMES	CO3 + CO5 (primary) · CO1 supporting
CONTACT & LAB LINK	8 lecture hours · supports Lab Experiments 7, 8
PRESCRIBED READING	How to Think Like a Computer Scientist: 8.1-8.8, 9.1, 11.1

AS PRESCRIBED (VERBATIM)

Modules: Random numbers, the time module, the math module, creating your own modules, Namespaces, Scope and lookup rules, Attributes and the dot Operator, Three import statement variants. Mutable versus immutable and aliasing. Object oriented programming: Classes and Objects — The Basics, Attributes, Adding methods to our class, Instances as arguments and parameters, Converting an instance to a string, Instances as return values. Chapter: 8.1-8.8, 9.1, 11.1 · Number of Hours: 8

4.1 Concepts in Focus

Modules

- **Random:** `random.random()` → [0,1) float; `randint(a, b)` → inclusive ints; `choice(seq)`, `shuffle(list)` (a modifier!). Seed it — `random.seed(7)` — to make lab output reproducible; the demo does, and the record should too.
- **Time:** `time.time()` (epoch float), `sleep(s)`, `strptime` for dates.
- **Math:** `sqrt`, `sin`, `cos`, `pi`, `e`, `factorial` — read from the module with a dot, not imported as bare names.
- **Creating your own:** any `.py` file is a module; `import mytools` executes it once and binds the name. Functions defined there are called as `mytools.f(x)`.
- **Three import variants** — the syllabus counts them, the viva asks them: ① `import math` → use `math.pi`; ② `from math import sqrt` → bare `sqrt`; ③ `from math import sqrt as square_root` → your own name. Variant ② floods the namespace, which is the next point.
- **Namespaces & scope (LEGB):** Local → (Enclosing) → Global → Built-in. A name is looked up at use, innermost outward; a function may read globals freely but must say `global x` to rebind one. Shadowing (`pi = 3` at module level) is legal — and a bug report waiting.
- **Attributes and the dot operator:** the dot is one idea at every scale — `module.function`, `object.attribute`, `s.find(...)` is a method lookup on a string object.
- **Mutable vs immutable and aliasing:** ints, floats, strings, tuples are immutable — aliasing them is harmless (no method can change them). Lists, dicts, class instances are mutable — aliasing them is sharing, and the shared mutable default (`def f(bag= [])`) is the trap the demo shows on purpose.

Classes & objects — basics

- `class Point:` defines a type; `p = Point()` makes an **instance**; data attributes are attached with `p.x = 3` — but the disciplined form is `__init__(self, x, y)`, the initialiser that sets state at creation.
- **Adding methods:** functions inside the class; the first parameter is **self** — the instance the method was called on. `p.length()` passes `p` as `self` implicitly.
- **Instances as arguments/parameters:** they pass by reference — a method or a plain function can mutate the caller's object (the difference pure functions vs modifiers, now with objects).
- **Converting an instance to a string:** define `__str__(self)` and `print(p)` becomes readable — Lab 8 and the open-ended tools all use it.

- **Instances as return values:** a function may build and return a new object (the “add returns a new Complex” clause of Lab 8) — the pure-function style in OOP clothing.

4.2 Worked Demo Program

The demo seeds randomness (so the numbers in the book are the numbers on screen), touches time and math, writes a tiny module to disk and imports it, exercises the three import variants, rebinds a global through a function, and detonates the mutable-default trap with the diagnosis in the transcript.

Run from the working directory, exactly as executed for this book:

```
python3 m04.py
```

m04.py · complete source (32 lines, max width 91 cols)

```
# Module 4 demo -- modules, namespaces & scope, import variants, aliasing
# Run : python3 m04.py
import math                      # variant 1: whole module
from math import factorial        # variant 2: one name
from math import sqrt as square_root # variant 3: renamed import
import random, time

random.seed(7)                   # deterministic dice, not luck
print("random.random(): %.6f | randint(1,6): %d" % (random.random(), random.randint(1, 6)))
print("math.pi: %.6f | time.time() is a float: %s" % (math.pi, type(time.time()) is float))
print("factorial(6):", factorial(6), "| sqrt(99): %.4f" % square_root(99))

# a module can also be CREATED from code and imported at run time:
with open("mytools.py", "w") as f:
    f.write("CELSIUS0 = 32\nFACTOR = 5 / 9\n\ndef f_to_c(t):\n"
           "    return (t - CELSIUS0) * FACTOR\n")
import mytools
print("created module mytools.f_to_c(98.6): %.2f C" % mytools.f_to_c(98.6))

counter = 0                      # global namespace
def bump():
    global counter
    counter += 1                 # without `global`, this is an error
bump(); bump()
print("\nglobal counter after two bump():", counter, "(lookup: local->global->built-in)")

def add_item(item, bag=[]):      # CLASSIC TRAP: mutable default
    bag.append(item)
    return bag
first = add_item("a")
second = add_item("b")
print("shared default list aliased:", first is second, "->", second)
```

4.3 Execution (Captured)

Captured terminal session — the program’s lines and the echoed input as typed at the bench:

Captured session

```
random.random(): 0.323833 | randint(1,6): 2
math.pi: 3.141593 | time.time() is a float: True
factorial(6): 720 | sqrt(99): 9.9499
created module mytools.f_to_c(98.6): 37.00 C

global counter after two bump(): 2 (lookup: local->global->built-in)
shared default list aliased: True -> ['a', 'b']
```

4.4 Quick-Reference Summary

Syllabus phrase	Python in one line
Random numbers	<code>random.seed(7); random.randint(1, 6)</code>

Syllabus phrase	Python in one line
creating your own modules	<code>write mytools.py; import mytools</code>
Three import variants	<code>import m · from m import f · from m import f as g</code>
Namespaces & lookup (LEGB)	local → enclosing → global → built-in
Attributes & the dot	one notation: <code>math.pi</code> , <code>s.strip()</code> , <code>p.x</code>
Classes & instances	<code>class P: def __init__(self, x): self.x = x</code>
Instances as args / returns	passed by reference; <code>return P(x+1)</code> is pure style
Converting an instance to a string	define <code>__str__</code> → readable print

4.5 Checkpoint Questions

1. List the three import variants with their call-site syntax, and state when each is the right choice.
2. Trace the lookup of name `counter` in: module defines it, function reads it, function assigns it without `global`. What error appears, and why is that a feature?
3. Explain the mutable-default trap: what is `first` is `second` after two calls to `def add_item(item, bag=[])`, and how do you fix the signature?
4. Why is aliasing harmless for tuples and strings but not for lists? Connect the answer to mutability, not to syntax.
5. Write the `Point` class with `__init__` and `__str__` so that `print(Point(3, 4))` reads “Point(3, 4)”.
6. Give one use each of `time.time()`, `random.choice()` and `math.floor()` that a lab record would accept.

Model answers to the most-asked questions

Q2. Trace the lookup of name `counter` in: module defines it, function reads it, function assigns it without `global`. What error appears, and why is that a feature?

Ans. Inside the function, the read of `counter` would find the module `global` — until the assignment makes `counter` local to the function for its entire body, so the read now precedes a local binding and Python raises `UnboundLocalError`. The error is the feature: silent reads of a `global` you are about to shadow would make every function a landmine; adding `global counter` states the intent.

Q4. Why is aliasing harmless for tuples and strings but not for lists? Connect the answer to mutability, not to syntax.

Ans. True — `bag` is created once when the `def` executes, so both calls share one list object: `first` is `second`. The fix is the sentinel idiom: `def add_item(item, bag=None): if bag is None: bag = []`.

4.6 Exercises & Practice

1. Build `stats.py` with mean, variance, std over a list (no NumPy); import it and verify against Lab 3a’s numbers on the same data.
2. Simulate 100 dice rolls with a frequency dict and `random.seed(42)`; print a table; then remove the seed and see which lines move.
3. Write class `BankAccount` (owner, balance) with `deposit/withdraw` that refuse negative results via `ValueError`; print an instance.
4. Time three summing styles (loop, `sum()`, NumPy) over `range(10**6)` with `time.perf_counter()`; record the winner and one sentence why.

Module 5 — Objects, Inheritance, Operator Overloading and Exceptions

Module 5 completes the OOP arc — objects change state and that matters, two names can mean one or two things, classes inherit and specialise, + learns new meanings — and closes the course where it started: errors, now as a control structure you choose: exceptions.

COURSE OUT-COMES	CO5 (primary) · CO1 + CO3 supporting
CONTACT & LAB LINK	8 lecture hours · supports Lab Experiments 7, 8, 10, 11
PRESCRIBED READING	How to Think Like a Computer Scientist: 11.2.2-11.2.4, 11.3.2-11.3.9, 12.1, 12.2

AS PRESCRIBED (VERBATIM)

Object oriented programming: Objects are mutable, Sameness, Copying. Inheritance: Pure functions, Modifiers, Generalization, Operator Overloading, Polymorphism. Exceptions: Catching Exceptions, Raising your own exceptions. Chapter: 11.2.2-11.2.4, 11.3.2-11.3.9, 12.1, 12.2 · Number of Hours: 8

5.1 Concepts in Focus

Objects are mutable · sameness · copying

- Instances mutate: a method call `a.deposit(50)` changes the one object every alias sees — the same sharing rule as lists, now carrying behaviour.
- **Sameness** has two tests: `a is b` (identity — one object) and `a == b` (equivalence — equal state; for classes without `__eq__`, `==` also degenerates to identity). A two-line experiment (alias, then deep-copy, then mutate through one name) proves all four combinations — it is the demo's first act.
- **Copying**: `copy.copy` shallow (new top-level, shared nested) vs `copy.deepcopy` (fully independent). For flat objects the difference is invisible; for an object holding a list it is everything.

Inheritance and its vocabulary

- `class Savings(Account):` — a **child class** gains all attributes and methods, `super().__init__()` chains construction, and redefining a method is **overriding**.
- **Pure functions vs modifiers** reappear as class design: a method that returns a new object (pure) vs one that changes the receiver (modifier). Saying which you wrote is the kind of sentence the rubric rewards.
- **Generalization**: pull what is common into a base class (state + shared methods), leave differences in the children — the reason `print` works on a mixed list of `Accounts` and `Savings`.
- **Operator overloading**: implement `__add__`, `__lt__`, `__str__` and Python's operators work on your class — Lab 8's "return a new complex" is exactly `__add__` written as a plain function.
- **Polymorphism**: one call site, many classes; `isinstance(x, Account)` stays True for a `Savings` object — inheritance is also an "is-a" contract the interpreter checks.

Exceptions — catching and raising

- **Catching**: `try: risky() except ValueError as e: handle(e) else: ... finally: ...` — except clauses are matched top-down; catch narrow (the named exception), not bare `except:`.
- **Raising your own**: `raise ValueError("price cannot be negative")` — the message is the diagnosis the examiner reads. The module's other tool, `assert`, states a developer's postulate (`assert a > 0`) — Lab 7 asks for both, in one function, and the book runs both failure paths.
- Traceback anatomy for these courses: exception type line last; the chain of calls above it; for `assert` the message appears after "AssertionError:".

5.2 Worked Demo Program

The demo mutates through an alias, deep-copies an Account and shows the copy untouched; probes `is` vs `==`; prints a mixed list of Accounts and Savings through each class's own `__str__` (polymorphism + generalization in three lines); overloads `+` on a Money class; and ends with a raised-and-caught `ValueError` whose message is the program's only output.

Run from the working directory, exactly as executed for this book:

```
python3 m05.py
```

m05.py · complete source (51 lines, max width 78 cols)

```
# Module 5 demo -- mutable objects, sameness, copying, inheritance, exceptions
# Run   : python3 m05.py
import copy

class Account:
    def __init__(self, owner, balance=0.0):
        self.owner = owner
        self.balance = balance
    def deposit(self, amount):
        self.balance += amount
    def __str__(self):
        return f"Account({self.owner}, Rs {self.balance:.2f})"

a = Account("Ravi", 100.0)
b = a                      # aliasing: two names, ONE object
c = copy.deepcopy(a)       # independent copy -- same state, different object
b.deposit(50)              # mutating through b is visible through a
print("after aliased deposit:", a)
print("deep copy untouched :", c)
print("sameness: a is b ->", a is b, "| a is c ->", a is c)

class Savings(Account):    # inheritance: reuse + specialize
    def __init__(self, owner, balance=0.0, rate=0.06):
        super().__init__(owner, balance)
        self.rate = rate
    def add_interest(self):
        self.balance *= (1 + self.rate)

s = Savings("Meera", 1000.0)
s.add_interest()
print("\npolymorphism -- print uses each class's __str__:")
for acct in (a, s):
    print("  ", acct, "| isinstance Account:", isinstance(acct, Account))

class Money:
    def __init__(self, rupees):
        self.rupees = rupees
    def __add__(self, other):          # operator overloading
        return Money(self.rupees + other.rupees)
    def __str__(self):
        return f"Rs {self.rupees}"
print("\nMoney(150) + Money(250) =", Money(150) + Money(250))

def apply_discount(price):
    if price < 0:
        raise ValueError("price cannot be negative")    # raising your own
    return price * 0.9
try:
    apply_discount(-10)
except ValueError as e:
    print("caught our own exception ->", e)
```

5.3 Execution (Captured)

Captured terminal session — the program's lines and the echoed input as typed at the bench:

Captured session

```

after aliased deposit: Account(Ravi, Rs 150.00)
deep copy untouched : Account(Ravi, Rs 100.00)
sameness: a is b -> True | a is c -> False

polymorphism -- print uses each class's __str__:
    Account(Ravi, Rs 150.00) | isinstance Account: True
    Account(Meera, Rs 1060.00) | isinstance Account: True

Money(150) + Money(250) = Rs 400
caught our own exception -> price cannot be negative

```

5.4 Quick-Reference Summary

Syllabus phrase	Python in one line
Objects are mutable	method call through any alias edits the shared state
Sameness	<code>a is b</code> (one object) vs <code>a == b</code> (equal state)
Copying	<code>copy.copy</code> shallow · <code>copy.deepcopy</code> fully separate
Inheritance	<code>class B(A):</code> · <code>super().__init__()</code> · override
Pure functions / modifiers	return new object vs mutate <code>self</code>
Generalization / polymorphism	shared behaviour in base; one print, many classes
Operator overloading	<code>def __add__(self, other):</code> return <code>Money(self.n + other.n)</code>
Catching exceptions	<code>try/except Name as e:</code> — narrow, ordered
Raising your own	<code>raise ValueError("why")</code> · <code>assert cond, "msg"</code>

5.5 Checkpoint Questions

1. Prove with two lines of code that aliasing and shallow copying behave differently on an object holding a list.
2. When is `__eq__` identity in disguise? What must a class define to make `==` about state?
3. Explain, with the Savings demo as witness, how generalization and polymorphism combine: which class's `__str__` runs, and why?
4. Contrast `assert a > 0` with `if a <= 0: raise ValueError(...)`: audience, message, and what each is for in a lab program.
5. Write the exception ladder for reading an integer from a file: `ValueError`, `FileNotFoundError`, then a general Exception — what does each catch mean, and why that order?
6. Lab 8 demands `add_complex(x, y)` returning a NEW Complex. State in one sentence which style (pure/modifier) that is, and how the same class would express it as overloading.

Model answers to the most-asked questions

Q4. Contrast `assert a > 0` with `if a <= 0: raise ValueError(...)`: audience, message, and what each is for in a lab program.

Ans. `assert` states a developer postulate — “if this is false, the program logic is broken” — optimised away with the `-O` flag, message owned by `AssertionError`; `raise ValueError` is a designed API decision for callers — it ships with the program always, carries a written diagnosis, and is what a record grader expects to see guarding user input. In Lab 7 the assertion covers `a>0` (the contract) and the raise covers `b==0` (the legitimate refusal), so each failure prints a different, named reason.

Q6. Lab 8 demands `add_complex(x, y)` returning a NEW Complex. State in one sentence which style (pure/modifier) that is, and how the same class would express it as overloading.

Ans. Pure — no receiver changes, the result is a fresh object; overloading expresses the same rule as `__add__` so that `x + y` compiles, while `add_complex(x, y)` remains the graded call form in Lab 8. The record should say both sentences: which style was chosen and what the syllabus asked for.

5.6 Exercises & Practice

1. Extend the demo's Money class with `__lt__` and `__eq__`; sort a list of Money — no `key=` argument allowed.
2. Model Rectangle (w, h) with `area()/perimeter()/__str__`, then Square as a subclass reusing `__init__` via `super`; `assert isinstance(s, Rectangle)`.
3. Wrap the Lab 1a calculator's division in `try/except ZeroDivisionError` and an explicit `raise` with a friendlier message; compare the two transcripts.
4. Give a class "Checked" an attribute bag (list). Deep-copy an instance, add to the original's bag; print both — then repeat with shallow copy and explain the difference in two sentences.

P A R T

B

Laboratory — Questions 1 to 12

The fixed set and the four open-ended tools, complete: the question verbatim, the approach, the full program (both halves of every a/b pair), captured transcripts that include the guard runs, the hand-verified numbers, viva questions with model answers, and extensions worth attempting after submission.

IN THIS PART

- Experiment 1 — menu calculator; senior-citizen test
- Experiment 2 — Fibonacci to N; list lifecycle
- Experiment 3 — mean/variance/std; digit frequencies
- Experiment 4 — 10 most frequent words in a file
- Experiment 5 — marks report by bubble sort
- Experiment 6 — sort a file's lines into a new file
- Experiment 7 — DivExp: assertion + raised exception
- Experiment 8 — Complex objects, pure adder, N-fold
- Experiment 9 — text analysis tool (open-ended)
- Experiment 10 — CSV summary generator (open-ended)
- Experiment 11 — student grade tracker (open-ended)
- Experiment 12 — recursive folder listing (open-ended)

Experiment 1 — Calculator by Choice, and a Senior-Citizen Test

One syllabus entry, two programs, one lesson: arithmetic of `input()`, the selection ladder, and a guard for every input the examiner might type by accident.

AIM	(a) Read 2 numbers and perform the basic arithmetic operations by choice (1-Add 2-Subtract 3-Multiply 4-Divide). (b) Read name and year of birth; display whether the person is a senior citizen.
OBJECTIVE	int/float conversion, if-elif ladder with an else, comparison operators, guarded division by zero, fixed reference year for reproducible age.
ENVIRONMENT & MATERIALS	Python 3.13 on the laboratory Linux machines — identical behaviour in IDLE, VS Code or any CPython 3.8+ on Windows. Only the standard library (plus NumPy in Module 3 notes). Every transcript below was captured from a real pseudo-terminal run of the source printed in the same chapter.
COURSE OUT-COMES	CO1 (see the CO map in the front matter)

AS PRESCRIBED (VERBATIM)

1 a. Develop a python program to read 2 numbers from the keyboard and perform the basic arithmetic operations based on the choice. (1-Add, 2-Subtract, 3-Multiply, 4-Divide). b. Develop a program to read the name and year of birth of a person. Display whether the person is a senior citizen or not.

1.1 Approach

- Convert once at the edge: `float(input(...))` — every later use is numeric.
- The choice is an `if-elif-elif-else` ladder over integer constants 1..4; the final else is “invalid choice”, not a silent pass.
- Division checks `b == 0` before dividing and stops with a message — the record must show this run, not just describe it.
- Part (b) fixes `CURRENT_YEAR = 2026` so the captured output never drifts between the day it was run and the day it is read; age is one subtraction, status is one comparison ladder (`<0` guard, `≥60` senior, else not).

1.2 Program Code

The programs below are the ones executed in §1.3: listing and transcript come from the same files.

lab01a.py · complete source (28 lines, max width 77 cols)

```
# Lab 1a -- two numbers, basic arithmetic by choice (1-Add 2-Sub 3-Mul 4-Div)
# Run   : python3 lab01a.py
print("=== Simple Calculator ===")
a = float(input("Enter first number : "))
b = float(input("Enter second number: "))
print("1-Add   2-Subtract  3-Multiply  4-Divide")
choice = int(input("Enter your choice : "))

if choice == 1:
    result = a + b
    sign = "+"
elif choice == 2:
    result = a - b
    sign = "-"
elif choice == 3:
    result = a * b
    sign = "*"
elif choice == 4:
    if b == 0:
        print("Error: Division by zero")
    else:
        result = a / b
        sign = "/"
else:
    print("Invalid choice")
```

```

        print("Error: division by zero is not defined.")
        raise SystemExit
    result = a / b
    sign = "/"
else:
    print("Invalid choice -- run again with 1..4.")
    raise SystemExit

print(f"{a:g} {sign} {b:g} = {result:g}")

```

lab01b.py · complete source (16 lines, max width 74 cols)

```

# Lab 1b -- name + birth year -> senior citizen? (60+ as per GoI norm)
# Run : python3 lab01b.py
CURRENT_YEAR = 2026      # fixed so the record's output is reproducible

name = input("Enter the name of the person      : ")
year = int(input("Enter the year of birth (YYYY) : "))
age = CURRENT_YEAR - year

print("Name:", name)
print(f"Age in {CURRENT_YEAR}: {age} years")
if age < 0:
    print("Check the input -- birth year lies in the future.")
elif age >= 60:
    print("Status: SENIOR CITIZEN")
else:
    print("Status: not a senior citizen")

```

1.3 Execution & Transcripts

Invocation, typed exactly as shown:

```
python3 lab01a.py  python3 lab01b.py
```

Captured terminal sessions follow — input appears on the same line as its prompt, echoed by the terminal, exactly as at the bench. Guard runs (bad input) are included on purpose: they are part of the evidence.

Run 1a-1 — multiply

```

=== Simple Calculator ===
Enter first number : 25
Enter second number: 10
1-Add  2-Subtract  3-Multiply  4-Divide
Enter your choice : 3
25 * 10 = 250

```

Run 1a-2 — the divide-by-zero guard

```

=== Simple Calculator ===
Enter first number : 48
Enter second number: 0
1-Add  2-Subtract  3-Multiply  4-Divide
Enter your choice : 4
Error: division by zero is not defined.

```

Run 1a-3 — invalid menu choice

```

=== Simple Calculator ===
Enter first number : 7
Enter second number: 5
1-Add  2-Subtract  3-Multiply  4-Divide
Enter your choice : 9
Invalid choice -- run again with 1..4.

```

Run 1b-1 — born 1958, age 68

```

Enter the name of the person      : Ramesh
Enter the year of birth (YYYY)   : 1958
Name: Ramesh
Age in 2026: 68 years
Status: SENIOR CITIZEN

```

Run 1b-2 — born 2000, age 26

```

Enter the name of the person      : Kavya
Enter the year of birth (YYYY)   : 2000
Name: Kavya
Age in 2026: 26 years
Status: not a senior citizen

```

Run 1b-3 — birth year in the future

```

Enter the name of the person      : Future Person
Enter the year of birth (YYYY)   : 2035
Name: Future Person
Age in 2026: -9 years
Check the input -- birth year lies in the future.

```

1.4 Observations & Analysis

The three calculator runs cover the three exits of the ladder (valid operation, refused division, out-of-range choice); the age runs cover the three exits of the comparison (senior, not senior, impossible). Note that `{a:g}` prints 250 without a trailing “.0” — format choices are part of the observable output.

Input	Path taken	Output
25, 10, choice 3	elif 3 → <code>a*b</code>	<code>25 * 10 = 250</code>
48, 0, choice 4	elif 4, <code>b==0</code> branch	Error message, <code>SystemExit</code>
Ramesh, 1958	<code>age ≥ 60</code>	SENIOR CITIZEN
person, 2035	<code>age < 0</code>	input check message

1.5 Result

Both programs were developed, executed and verified: all four arithmetic operations work through the menu with division guarded against zero, and the senior-citizen decision is correct for the boundary age 60, ordinary ages and invalid future years. CO1 achieved.

1.6 Viva Voce Questions

1. Why `float(input(...))` and not `int(input(...))` in the calculator?
2. What would `print('25 * 10 =', a*b)` output — is the format string necessary?
3. Where exactly does the program stop if `b` is zero — which statement ends it?
4. How would you make the senior-citizen check boundary-inclusive at 60? Quote the line.
5. State one semantic error each program is guarded against.

Model answers to the most-asked questions

Q3. Where exactly does the program stop if `b` is zero — which statement ends it?

Ans. `print('25 * 10 =', a*b)` gives `'25 * 10 = 250.0'` — the comma form shows the raw float. The f-string with `:g` renders 250, which is why record-friendly output uses it; both are syntactically correct, only one prints like a calculator.

Q5. State one semantic error each program is guarded against.

Ans. The calculator guards the semantic error of trusting `input()`: a string entering arithmetic would raise `TypeError`, so conversion happens immediately and the choice is validated. The age program

guards the impossible input (birth year in the future) that would otherwise print 'Age: -9 years, not a senior citizen' — an answer that is confidently wrong.

1.7 Exercises & Extensions

1. Extend the menu with modulus and integer division (choices 5, 6) and re-verify all six paths.
2. Allow the year of birth to be a string like '1958' or an int — make both work without try/except.
3. Rewrite (b) to use the real current year via `datetime.date.today().year` and discuss why the captured transcript in this book pins 2026 instead.
4. Add a loop so the calculator repeats until the user chooses 0; identify which variable is the loop sentinel.

Experiment 2 — Fibonacci to Length N, and the Full List Lifecycle

Two halves of Module 1-2 in one question: a while-free `for` that updates a pair of variables, and a list walked from creation to `clear()`.

AIM	(a) Generate the Fibonacci sequence of length N, N read from the console. (b) Create a list and perform insert, remove, append, length, pop and clear.
OBJECTIVE	tuple assignment as the update step, list as an accumulator, the exact semantics of each list method (value vs index removal, pop returns).
ENVIRONMENT & MATERIALS	Python 3.13 on the laboratory Linux machines — identical behaviour in IDLE, VS Code or any CPython 3.8+ on Windows. Only the standard library (plus NumPy in Module 3 notes). Every transcript below was captured from a real pseudo-terminal run of the source printed in the same chapter.
COURSE OUT-COMES	CO1 + CO2 (see the CO map in the front matter)

AS PRESCRIBED (VERBATIM)

2 a. Develop a program to generate Fibonacci sequence of length (N). Read N from the console. b. Write a python program to create a list and perform the following operations: Inserting an element, Removing an element, Appending an element, Displaying the length of the list, Popping an element, Clearing the list.

2.1 Approach

- Fibonacci: hold the pair $(a, b) = (0, 1)$; append `a`, then `a, b = b, a + b` — one line, no temporary, because tuple assignment evaluates the right side first.
- Validate N once at the top; $N \leq 0$ prints a message instead of an empty “sequence”.
- For (b), print the list after every operation — the difference between insert-at-index and append-at-end, remove-by-value and pop-by-index, becomes visible in the transcript itself.

2.2 Program Code

The programs below are the ones executed in §2.3: listing and transcript come from the same files.

lab02a.py · complete source (13 lines, max width 76 cols)

```
# Lab 2a -- Fibonacci sequence of length N read from the console
# Run   : python3 lab02a.py
n = int(input("How many Fibonacci numbers? "))
if n <= 0:
    print("Please enter a positive integer.")
else:
    a, b = 0, 1                # tuple assignment does the swap in one line
    seq = []
    for _ in range(n):
        seq.append(a)
        a, b = b, a + b
    print(f"Fibonacci sequence of length {n}:")
    print(", ".join(str(x) for x in seq))
```

lab02b.py · complete source (23 lines, max width 71 cols)

```
# Lab 2b -- list lifecycle: insert, remove, append, len, pop, clear
# Run   : python3 lab02b.py
fruits = ["mango", "apple", "guava"]
print("initial      :", fruits)

fruits.insert(1, "jackfruit")    # position matters with insert
print("insert at 1  :", fruits)
```

```

fruits.remove("apple")           # remove by VALUE, not by index
print("remove 'apple':", fruits)

fruits.append("banana")         # add at the end
print("append banana :", fruits)

print("length      :", len(fruits))

popped = fruits.pop(2)          # pop RETURNS what it removes
print(f"pop(2) took {popped!r} ->", fruits)
fruits.pop()                    # no index: the last item
print("pop last     :", fruits)

fruits.clear()
print("clear()      :", fruits, "| length now:", len(fruits))

```

2.3 Execution & Transcripts

Invocation, typed exactly as shown:

```
python3 lab02a.py  python3 lab02b.py
```

Captured terminal sessions follow — input appears on the same line as its prompt, echoed by the terminal, exactly as at the bench. Guard runs (bad input) are included on purpose: they are part of the evidence.

Run 2a-1 — length 10

```

How many Fibonacci numbers? 10
Fibonacci sequence of length 10:
0, 1, 1, 2, 3, 5, 8, 13, 21, 34

```

Run 2a-2 — the N=1 edge

```

How many Fibonacci numbers? 1
Fibonacci sequence of length 1:
0

```

Run 2a-3 — rejected N=0

```

How many Fibonacci numbers? 0
Please enter a positive integer.

```

Run 2b-1 — every operation, printed

```

initial      : ['mango', 'apple', 'guava']
insert at 1   : ['mango', 'jackfruit', 'apple', 'guava']
remove 'apple': ['mango', 'jackfruit', 'guava']
append banana : ['mango', 'jackfruit', 'guava', 'banana']
length       : 4
pop(2) took 'guava' -> ['mango', 'jackfruit', 'banana']
pop last     : ['mango', 'jackfruit']
clear()      : [] | length now: 0

```

2.4 Observations & Analysis

Length 10 yields 0,1,1,2,3,5,8,13,21,34 — ten terms starting at F(0). The pair update never uses a temp variable; removing 'apple' (a value) and popping index 2 (a position, which returned 'guava' and left the list one shorter) are the two removal semantics the viva separates. `clear()` leaves `[]` with length 0, and the list object itself still exists — clearing is not deletion.

2.5 Result

The Fibonacci generator produces the correct sequence for $N = 10$ and $N = 1$, rejects $N \leq 0$ with a message, and the list program demonstrates all six required operations with the list state printed after each. CO1 and CO2 evidenced.

2.6 Viva Voce Questions

1. Why does `a, b = b, a+b` need no temporary variable?
2. What does `pop(2)` return and what remains in the list — give both from the transcript.
3. Distinguish `remove('apple')` from `pop(2)` in one sentence each.
4. If the list were cloned before `clear()`, would the clone be affected? Name the module-2 concept.
5. Why is the empty sequence for `N=0` not printed as a blank line by this program?

Model answers to the most-asked questions

Q4. If the list were cloned before `clear()`, would the clone be affected? Name the module-2 concept.

Ans. No — this list is not shared: nothing aliased it. But in general `clear()` is a modifier, so every alias sees the emptied list; a clone taken before `clear()` survives untouched. This is exactly the aliasing/clone pair of Module 2.

Q5. Why is the empty sequence for `N=0` not printed as a blank line by this program?

Ans. Because `N≤0` takes the guard branch that prints 'Please enter a positive integer.' and never enters the loop — the program distinguishes 'sequence of length zero' (a valid request it declines) from silently printing an empty line.

2.7 Exercises & Extensions

1. Print Fibonacci numbers not exceeding a limit `L` instead of a count `N` (while loop).
2. Add slice and membership checks to 2b: first two elements, and whether 'kiwi' is in the list before and after `append`.
3. Build the same list output using only pure operations (slices and concatenation) and compare line counts.
4. Generate Fibonacci in a tuple (not list); state what changes when you try `nums[0] = 1`.

Experiment 3 — Mean, Variance, Standard Deviation — and Digit Frequencies

The console-to-statistics pipeline: one list built from N inputs and reduced with the formulas, the other read as characters and counted with a dictionary before dictionaries are even in the module — numbers as strings are still strings.

AIM	(a) Read N numbers into a list; print mean, variance and standard deviation with suitable messages. (b) Read a multi-digit number as chars; print the frequency of each digit.
OBJECTIVE	accumulator loops, the variance formula as a generator expression, str-iteration for digit counting, dict.get for safe counting, input validation by str.isdigit.
ENVIRONMENT & MATERIALS	Python 3.13 on the laboratory Linux machines — identical behaviour in IDLE, VS Code or any CPython 3.8+ on Windows. Only the standard library (plus NumPy in Module 3 notes). Every transcript below was captured from a real pseudo-terminal run of the source printed in the same chapter.
COURSE OUT-COMES	CO1 + CO2 (see the CO map in the front matter)

AS PRESCRIBED (VERBATIM)

3 a. Read N numbers from the console and create a list. Develop a program to print mean, variance and standard deviation with suitable messages. b. Read a multi-digit number (as chars) from the console. Develop a program to print the frequency of each digit with a suitable message.

3.1 Approach

- (a) $\text{mean} = \text{sum}(\text{data})/n$; $\text{variance} = \sum(x - \text{mean})^2/n$ — written as one generator expression; $\text{std} = \text{variance}^{**0.5}$ (no math import needed — `x ** 0.5` is the course idiom here).
- Print all four decimals (`{:.4f}`) — “suitable messages” includes labelled, aligned output.
- (b) The number is a string; iterate it character by character; a dict with `freq[d] = freq.get(d, 0) + 1` counts; digits-only validation before counting (`raw.lstrip("-").isdigit()`), negative sign accepted and ignored.

3.2 Program Code

The programs below are the ones executed in §3.3: listing and transcript come from the same files.

lab03a.py · complete source (15 lines, max width 74 cols)

```
# Lab 3a -- N numbers -> mean, variance, standard deviation
# Run   : python3 lab03a.py
n = int(input("How many numbers? "))
data = []
for i in range(n):
    data.append(float(input(f"Number {i + 1}: ")))

mean = sum(data) / n
variance = sum((x - mean) ** 2 for x in data) / n    # population variance
std = variance ** 0.5

print("\nData set:", data)
print(f"Mean           = {mean:.4f}")
print(f"Variance        = {variance:.4f}")
print(f"Standard deviation = {std:.4f}")
```

lab03b.py · complete source (17 lines, max width 78 cols)

```
# Lab 3b -- multi-digit number read AS CHARS -> frequency of each digit
# Run   : python3 lab03b.py
raw = input("Enter a multi-digit number: ").strip()
if not raw.lstrip("-").isdigit():
    print("That is not a whole number -- check the input.")
```

```

    raise SystemExit

digits = raw.lstrip("-")
freq = {}
for ch in digits:
    # iterating the string, one char at a time
    freq[ch] = freq.get(ch, 0) + 1

print(f"\nFrequency of each digit in {raw}:")
for d in sorted(freq):
    times = "time" if freq[d] == 1 else "times"
    print(f"  digit {d} appears {freq[d]} {times}")
print("Total digits counted:", len(digits))

```

3.3 Execution & Transcripts

Invocation, typed exactly as shown:

```
python3 lab03a.py    python3 lab03b.py
```

Captured terminal sessions follow — input appears on the same line as its prompt, echoed by the terminal, exactly as at the bench. Guard runs (bad input) are included on purpose: they are part of the evidence.

Run 3a-1 — 10..50, the classic data set

```

How many numbers? 5
Number 1: 10
Number 2: 20
Number 3: 30
Number 4: 40
Number 5: 50

Data set: [10.0, 20.0, 30.0, 40.0, 50.0]
Mean      = 30.0000
Variance  = 200.0000
Standard deviation = 14.1421

```

Run 3a-2 — all values equal ⇒ zero spread

```

How many numbers? 4
Number 1: 5
Number 2: 5
Number 3: 5
Number 4: 5

Data set: [5.0, 5.0, 5.0, 5.0]
Mean      = 5.0000
Variance  = 0.0000
Standard deviation = 0.0000

```

Run 3b-1 — 12332211

```

Enter a multi-digit number: 12332211

Frequency of each digit in 12332211:
  digit 1 appears 3 times
  digit 2 appears 3 times
  digit 3 appears 2 times
Total digits counted: 8

```

Run 3b-2 — single occurrences stated in singular

```

Enter a multi-digit number: 90210

Frequency of each digit in 90210:
  digit 0 appears 2 times
  digit 1 appears 1 time
  digit 2 appears 1 time

```

```
digit 9 appears 1 time
Total digits counted: 5
```

Run 3b-3 — rejected non-numeric

```
Enter a multi-digit number: 12a4
That is not a whole number -- check the input.
```

3.4 Observations & Analysis

For 10,20,30,40,50: mean 30.0000, variance 200.0000, $\sigma = 14.1421$ — the hand value $\sqrt{200}$ to the letter. The constant data set [5,5,5,5] gives variance 0, the “no spread” check the record must show. Digit counting on 12332211 returns 1×3 , 2×3 , 3×2 — 8 digits, and the total line confirms the loop saw every character exactly once; 90210 prints “1 time” (singular), so the message adapts. Non-digit input exits with a message — no traceback in a lab transcript.

3.5 Result

Both programs were executed and verified against hand calculations: the statistics trio is exact for the sample data set and correctly reports zero variance for constant data; digit frequency with plural handling and input validation completes question 3. CO1, CO2.

3.6 Viva Voce Questions

1. Why does the program treat the number as a string and not convert to int in (b)?
2. Write the variance line without a generator expression; how many extra lines does it cost?
3. What does `.get(d, 0)` protect against? Give the failing alternative.
4. If the input were 1234567890, how many dict entries exist at the end — justify without running.
5. State which formula (population vs sample variance) the program uses and where the difference would appear.

Model answers to the most-asked questions

Q1. Why does the program treat the number as a string and not convert to int in (b)?

Ans. Because the task is about digits, not magnitude: int conversion would drop the leading-zero case and buys nothing — iterating the raw string gives single characters directly, which are exactly the dict keys wanted. ('As chars' in the question is an instruction to keep it text.)

Q5. State which formula (population vs sample variance) the program uses and where the difference would appear.

Ans. Population variance: divide by n. Sample variance divides by $n-1$; with five numbers the value would be 250 instead of 200 ($\sigma = 15.8114$). The record should state which one it implemented — this one states: /n, population.

3.7 Exercises & Extensions

1. Add median to 3a (sort a copy, take the middle) and state whether sorting was a modifier or a copy-first pure step.
2. Extend 3b to count digits of a large number read with separators (1,23,322) — clean first, then count.
3. Compare your variance line with `statistics.pvariance` on the same list; print both and any float difference.
4. Rewrite 3a to read until a blank line instead of N first; keep identical output formatting.

Experiment 4 — Ten Most Frequent Words in a Text File

The dictionary's home ground, run exactly as the hint prescribes: count into a dict, sort by value descending, take the first ten — with a file as the source of the words.

AIM	Develop a program to print the 10 most frequently appearing words in a text file, using a dictionary of words and frequencies, sorted in reverse order of frequency, displaying the dictionary slice of the first 10 items.
OBJECTIVE	file read + token cleaning, dict counting idiom, sorted() with a key lambda, slice [:10], argument via sys.argv.
ENVIRONMENT & MATERIALS	Python 3.13 on the laboratory Linux machines — identical behaviour in IDLE, VS Code or any CPython 3.8+ on Windows. Only the standard library (plus NumPy in Module 3 notes). Every transcript below was captured from a real pseudo-terminal run of the source printed in the same chapter.
COURSE OUT-COMES	CO2 + CO4 (see the CO map in the front matter)

AS PRESCRIBED (VERBATIM)

4. Develop a program to print 10 most frequently appearing words in a text file. [Hint: Use a dictionary with distinct words and their frequency of occurrences. Sort the dictionary in the reverse order of frequency and display the dictionary slice of the first 10 items.]

4.1 Approach

- One helper `words_from(path)`: read, split on whitespace, strip punctuation from each token, lower-case, drop empties — three list-friendly string methods doing the dirty work.
- Count with the get-idiom (the hint's "dictionary with distinct words").
- `sorted(freq.items(), key=lambda kv: (-kv[1], kv[0]))`: frequency descending, ties alphabetical — deterministic output, which the record needs.
- Return `ranked[:10]`: the "dictionary slice of the first 10 items", printed in an aligned rank/count table.

4.2 Program Code

The programs below are the ones executed in §4.3: listing and transcript come from the same files.

lab04.py · complete source (29 lines, max width 78 cols)

```
# Lab 4 -- 10 most frequent words in a text file
# Run : python3 lab04.py ../data/paragraph.txt
import sys
from string import punctuation

def words_from(path):
    with open(path) as f:
        text = f.read()
    out = []
    for token in text.split():
        word = token.strip(punctuation + '"\'').lower()
        if word:
            out.append(word)
    return out

def top_n(path, n=10):
    freq = {}
    for w in words_from(path):
        freq[w] = freq.get(w, 0) + 1
    ranked = sorted(freq.items(), key=lambda kv: (-kv[1], kv[0]))
    return ranked[:n]

if len(sys.argv) != 2:
```

```

print("Usage: python3 lab04.py <textfile>")
raise SystemExit

print("rank  count  word")
for i, (word, count) in enumerate(top_n(sys.argv[1]), 1):
    print(f"{i:>3d}.    {count:>3d}    {word}")

```

4.3 Execution & Transcripts

Invocation, typed exactly as shown:

```
python3 lab04.py
```

Captured terminal sessions follow — input appears on the same line as its prompt, echoed by the terminal, exactly as at the bench. Guard runs (bad input) are included on purpose: they are part of the evidence.

Run 1 — the paragraph of data/paragraph.txt

```

rank  count  word
1.     6  python
2.     5   and
3.     4 errors
4.     3    a
5.     3 every
6.     3 programming
7.     3    to
8.     2   code
9.     2  easy
10.    2    is

```

Run 2 — a small file: all counts one, alphabetical

```

rank  count  word
1.     1  apple
2.     1 banana
3.     1  cake
4.     1  cherry
5.     1  date
6.     1  elder
7.     1  foster
8.     1  guava
9.     1   jam
10.    1   pie

```

4.4 Observations & Analysis

On the paragraph, python leads with 6 (the file uses it as both cases; lower- casing merges them), and 5, errors 4. Run 2 is the honesty check: a file of 12 distinct words has every count 1 — the tie-break alone decides the order, and it matches the alphabetical tie-break exactly. Note the strip step: "code," and "code." both become "code"; without it, counts would fragment.

4.5 Result

The program counts, ranks and slices per the prescribed hint and prints a stable, aligned top-10 table for any text file passed as argument; verified on two files including the all-counts-equal edge. CO2, CO4.

4.6 Viva Voce Questions

1. Which single line would you delete to show the whole frequency dictionary, not just ten?
2. Why lower() before counting, and what breaks without it? Quote the run's numbers.
3. Explain the key lambda's tuple — why minus the count, why the word as second element?
4. What happens if the file name is wrong: which exception, and where could you catch it?
5. items() returns what kind of object, and why does sorted() accept it?

Model answers to the most-asked questions

Q3. Explain the key lambda's tuple — why minus the count, why the word as second element?

Ans. (-count, word): sorted() is ascending, so negating the count flips frequency to descending; the word (ascending) then breaks ties alphabetically. One tuple, two sort orders — and the deterministic table the record shows.

Q5. items() returns what kind of object, and why does sorted() accept it?

Ans. A list of (key, value) tuples — dict.items() is a view of pairs, and sorted() consumes any iterable, producing a list of tuples that unpacks cleanly in the for-loop header.

4.7 Exercises & Extensions

1. Restrict to words of length ≥ 4 and re-run; report how many ranks changed.
2. Write the frequencies back to freq_out.txt as 'word,count' lines — then read that file with the Lab-10 style CSV loader.
3. Replace the lambda sort with a plain nested-loop bubble sort over pairs and compare runtime on a 1000-word file.
4. Catch the missing-file case with try/except (Module 5 arrives early here) and print the friendly message.

Experiment 5 — Marks Report by Bubble Sort (Highest → Lowest)

The one lab question that names its algorithm. The data is six marks; the lesson is that sorting parallel arrays means sorting the indices, and that a swap counter turns the routine into observable evidence.

AIM	Read 6 subject marks from the keyboard; display marks from the highest to the lowest score, reading them into a 1-D array and sorting using the Bubble Sort technique.
OBJECTIVE	nested for loops, comparison swap, descending order, index permutation to keep subject names attached, swap-count as a complexity witness.
ENVIRONMENT & MATERIALS	Python 3.13 on the laboratory Linux machines — identical behaviour in IDLE, VS Code or any CPython 3.8+ on Windows. Only the standard library (plus NumPy in Module 3 notes). Every transcript below was captured from a real pseudo-terminal run of the source printed in the same chapter.
COURSE OUT-COMES	CO1 + CO2 (see the CO map in the front matter)

AS PRESCRIBED (VERBATIM)

5. Develop a program to read 6 subject marks from the keyboard for a student. Generate a report that displays the marks from the highest to the lowest score attained by the student. [Read the marks into a 1-Dimensional array and sort using the Bubble Sort technique].

5.1 Approach

- Two parallel lists would drift out of sync; instead `order = list(range(6))` and the comparator looks up `marks[order[j]]` — the subjects ride along for free.
- Bubble pass: compare neighbours, swap if the left is smaller (descending); after pass `i`, the tail of `i+1` entries is final, so the inner loop shrinks.
- Count every swap — the transcript then reports “5 swap(s)”, real numbers on the effort, and Run 2’s all-equal data set proves the strict `<` comparator does zero work.

5.2 Program Code

The programs below are the ones executed in §5.3: listing and transcript come from the same files.

lab05.py · complete source (21 lines, max width 76 cols)

```
# Lab 5 -- six subject marks -> report highest to lowest (bubble sort)
# Run   : python3 lab05.py
subjects = ["Maths", "Physics", "Chemistry", "Python", "DSA", "English"]
marks = []
for s in subjects:
    marks.append(int(input(f"Enter marks for {s:<10}: ")))

order = list(range(6))          # sort INDICES so subject names ride along
swaps = 0
for i in range(5):              # ---- bubble sort, descending on marks ----
    for j in range(5 - i):
        if marks[order[j]] < marks[order[j + 1]]:
            order[j], order[j + 1] = order[j + 1], order[j]
            swaps += 1

print("\n===== MARKS REPORT (highest -> lowest) =====")
print("Rank   Subject      Marks")
for rank, k in enumerate(order, 1):
    print(f"{rank:>3d}.    {subjects[k]:<11}{marks[k]:>4d}")
print(f"\nTotal = {sum(marks)} | Average = {sum(marks) / 6:.2f}")
print(f"Bubble sort finished after {swaps} swap(s).")
```

5.3 Execution & Transcripts

Invocation, typed exactly as shown:

```
python3 lab05.py
```

Captured terminal sessions follow — input appears on the same line as its prompt, echoed by the terminal, exactly as at the bench. Guard runs (bad input) are included on purpose: they are part of the evidence.

Run 1 — six different marks

```
Enter marks for Maths      : 78
Enter marks for Physics    : 92
Enter marks for Chemistry  : 65
Enter marks for Python     : 88
Enter marks for DSA        : 49
Enter marks for English    : 71

===== MARKS REPORT (highest -> lowest) =====
Rank  Subject      Marks
1.    Physics      92
2.    Python       88
3.    Maths        78
4.    English      71
5.    Chemistry    65
6.    DSA          49

Total = 443 | Average = 73.83
Bubble sort finished after 5 swap(s).
```

Run 2 — all marks equal: no swaps

```
Enter marks for Maths      : 40
Enter marks for Physics    : 40
Enter marks for Chemistry  : 40
Enter marks for Python     : 40
Enter marks for DSA        : 40
Enter marks for English    : 40

===== MARKS REPORT (highest -> lowest) =====
Rank  Subject      Marks
1.    Maths        40
2.    Physics      40
3.    Chemistry    40
4.    Python       40
5.    DSA          40
6.    English      40

Total = 240 | Average = 40.00
Bubble sort finished after 0 swap(s).
```

5.4 Observations & Analysis

Input 78, 92, 65, 88, 49, 71 ranks Physics 92 → DSA 49; the report prints rank, subject, mark, total and average. Run 2 is the quiet proof of stability: five passes, zero swaps, original subject order preserved under equal marks. Worst case on 6 items is 15 comparisons; the printed swap count (5) is what an examiner checks against a hand trace.

5.5 Result

The report was generated correctly for distinct and identical mark sets; the bubble sort is verified by the swap count, and the descending order by the rank table. CO1, CO2.

5.6 Viva Voce Questions

1. Trace one pass by hand for 78 92 65 ... — which swaps fire and where does 49 end up after pass 1?

2. Why sort the order list instead of `marks.sort(reverse=True)`? The question says bubble sort — which part satisfies it?
3. What makes this sort stable in Run 2? Name the comparator property.
4. If the inner loop ran to `j = 5` instead of `5-i`, would the output change? What costs would rise?
5. Extend the report with grade letters — which line of this program already gives you the sorted list to walk?

Model answers to the most-asked questions

Q2. Why sort the order list instead of `marks.sort(reverse=True)`? The question says bubble sort — which part satisfies it?

Ans. Because sorting marks directly would sever the marks from their subject names; an index permutation is the classic answer, and the syllabus demands the bubble routine — the nested loop with neighbour swaps below satisfies that literally, while `sort()` would satisfy only the result, not the method.

Q3. What makes this sort stable in Run 2? Name the comparator property.

Ans. The strict `<` comparator: equal marks never trigger a swap, so their relative order survives — a stable sort. With `<=` the equal entries would churn and the output order would change for Run 2.

5.7 Exercises & Extensions

1. Re-run with marks as floats and print one decimal; confirm formatting only changes the table width.
2. Add a search: after sorting, find the rank of a subject entered by the user (linear scan is fine).
3. Implement the same report using `sorted(range(6), key=...)` and compare line count; state which is which in pure/modifier terms.
4. Count comparisons as well as swaps and print the triangular number 15 for six entries.

Experiment 6 — Sort a File’s Lines into a Separate Sorted File

Module 3’s file grammar in a single honest pass: read, clean, sort, write — with the hint’s five method names appearing exactly once each, in order.

AIM	Sort the contents of a text file and write the sorted contents into a separate text file, using <code>strip()</code> , <code>len()</code> , <code>sort()</code> , <code>append()</code> , <code>open()</code> , <code>readlines()</code> and <code>write()</code> .
OBJECTIVE	<code>open/read/write</code> cycles with <code>with</code> -blocks, line cleaning, <code>sorted()</code> vs <code>.sort()</code> , constructing the output path, verifying by printing what was written.
ENVIRONMENT & MATERIALS	Python 3.13 on the laboratory Linux machines — identical behaviour in IDLE, VS Code or any CPython 3.8+ on Windows. Only the standard library (plus NumPy in Module 3 notes). Every transcript below was captured from a real pseudo-terminal run of the source printed in the same chapter.
COURSE OUT-COMES	CO2 + CO4 (see the CO map in the front matter)

AS PRESCRIBED (VERBATIM)

6. Develop a program to sort the contents of a text file and write the sorted contents into a separate text file. [Hint: Use string methods `strip()`, `len()`, list methods `sort()`, `append()`, and file methods `open()`, `readlines()`, and `write()`].

6.1 Approach

- `f.readlines()` → list of raw lines; a comprehension strips each and drops blank ones (that is where `len()` gets used, reporting the count).
- One in-place `lines.sort()` — the modifier form the hint asks for; the file order is lexicographic, so “date Rolls” precedes “elder cake” by the first letter, capitals included.
- Write with an explicit loop of `f.write(ln + "\n")`: the newline is the program’s responsibility — the one thing that silently corrupts sorted files when forgotten.
- Print the result block in the transcript, so the record shows file contents without a second cat.

6.2 Program Code

The programs below are the ones executed in §6.3: listing and transcript come from the same files.

lab06.py · complete source (22 lines, max width 84 cols)

```
# Lab 6 -- sort the lines of a text file into a separate sorted file
# Run   : python3 lab06.py ../data/lines.txt
import os

name = input("Input file name: ").strip()
try:
    with open(name) as f:
        lines = [ln.strip() for ln in f.readlines() if ln.strip()]
except FileNotFoundError:
    print(f"'{name}' not found -- create it and retry.")
    raise SystemExit

lines.sort()
out = os.path.join(os.path.dirname(name) or ".", "sorted_" + os.path.basename(name))
with open(out, "w") as f:
    for ln in lines:
        f.write(ln + "\n")

print(f"Read {len(lines)} non-blank line(s) from {name}.")
print(f"Sorted content written to {out}:")
for ln in lines:
    print("    ", ln)
```

6.3 Execution & Transcripts

Invocation, typed exactly as shown:

```
python3 lab06.py
```

Captured terminal sessions follow — input appears on the same line as its prompt, echoed by the terminal, exactly as at the bench. Guard runs (bad input) are included on purpose: they are part of the evidence.

Run 1 — data/lines.txt → data/sorted_lines.txt

```
Input file name: ../data/lines.txt
Read 6 non-blank line(s) from ../data/lines.txt.
Sorted content written to ../data/sorted_lines.txt:
    apple pie
    banana foster
    cherry tart
    date Rolls
    elder cake
    guava jam
```

6.4 Observations & Analysis

Six non-blank lines read, six sorted and written to sorted_lines.txt beside the original (path built with os.path so any input directory works). If the input file is missing the program prints a message and exits — FileNotFoundError is caught, not raised to the screen.

6.5 Result

The sorted contents were written to the separate output file and verified by the printed listing; blank-line filtering and explicit newlines keep the output well-formed. CO4.

6.6 Viva Voce Questions

1. Why not just text.splitlines() then sort? What does the hint force you to demonstrate?
2. What does the file look like if you forget + '\n' in the write loop?
3. lines.sort() returns None — how would you have gotten the sorted list differently?
4. Where in this program is the one risk of overwriting data, and why is it safe here?
5. How would you make the sort case-insensitive? One keyword argument, one line.

Model answers to the most-asked questions

Q3. lines.sort() returns None — how would you have gotten the sorted list differently?

Ans. None — it is an in-place modifier. The pure alternative is sorted(lines), which returns a new list; the record says so when it names which style the program uses.

Q4. Where in this program is the one risk of overwriting data, and why is it safe here?

Ans. Opening the output with mode 'w' truncates whatever is there — the risk of writing. It is safe because the name is always prefixed sorted_ in the same directory, never equal to the input name.

6.7 Exercises & Extensions

1. Append mode: run twice with 'a' instead of 'w' and explain the duplicated lines the second time.
2. Sort case-insensitively and reverse=True; print both orders side by side.
3. Write the output as number-tab-line pairs, then read it back with the Lab-10 loader.
4. Extend: remove duplicate lines with one extra statement; state which module-2 structure does the job in a line.

Experiment 7 — DivExp with an Assertion and a Raised Exception

The question's two failure modes are not decoration — they are the answer. One function, one assert on $a > 0$, one raise on $b = 0$, one caller that catches both and prints the reason.

AIM	Define DivExp(a, b) returning $c = a/b$; write a suitable assertion for $a > 0$; raise an exception when $b = 0$; drive it from console values.
OBJECTIVE	assert as contract, raise with a diagnostic message, try/except with named exception classes, clean caller design.
ENVIRONMENT & MATERIALS	Python 3.13 on the laboratory Linux machines — identical behaviour in IDLE, VS Code or any CPython 3.8+ on Windows. Only the standard library (plus NumPy in Module 3 notes). Every transcript below was captured from a real pseudo-terminal run of the source printed in the same chapter.
COURSE OUT-COMES	CO1 + CO5 (see the CO map in the front matter)

AS PRESCRIBED (VERBATIM)

7. Develop a function named DivExp which takes TWO parameters a, b, and returns a value c ($c = a/b$). Write a suitable assertion for $a > 0$ in the function DivExp and raise an exception for when $b = 0$. Develop a suitable program that reads two console values and calls the function DivExp.

7.1 Approach

- `assert a > 0, "assertion failed: a must be positive"` — the assertion line is written exactly as the question demands, with the message the record can quote.
- `b == 0` is a legitimate caller error, not a program bug: `raise ZeroDivisionError("DivExp refuses b = 0")` hands the caller a named, caught path.
- The caller prints c on success and the exception name + message on failure — three runs, three outcomes, all in the transcript.

7.2 Program Code

The programs below are the ones executed in §7.3: listing and transcript come from the same files.

lab07.py · complete source (17 lines, max width 75 cols)

```
# Lab 7 -- DivExp(a, b) with assertion a > 0 and a raised error when b == 0
# Run   : python3 lab07.py
def DivExp(a, b):
    assert a > 0, "assertion failed: a must be positive"
    if b == 0:
        raise ZeroDivisionError("DivExp refuses b = 0")
    return a / b

a = float(input("Enter value of a: "))
b = float(input("Enter value of b: "))
try:
    c = DivExp(a, b)
    print(f"c = a / b = {c:g}")
except AssertionError as e:
    print("AssertionError:", e)
except ZeroDivisionError as e:
    print("ZeroDivisionError:", e)
```

7.3 Execution & Transcripts

Invocation, typed exactly as shown:

```
python3 lab07.py
```

Captured terminal sessions follow — input appears on the same line as its prompt, echoed by the terminal, exactly as at the bench. Guard runs (bad input) are included on purpose: they are part of the evidence.

Run 1 — 12 / 4: the happy path

```
Enter value of a: 12
Enter value of b: 4
c = a / b = 3
```

Run 2 — b = 0: the raised exception caught

```
Enter value of a: 12
Enter value of b: 0
ZeroDivisionError: DivExp refuses b = 0
```

Run 3 — a = -3: the assertion caught

```
Enter value of a: -3
Enter value of b: 4
AssertionError: assertion failed: a must be positive
```

7.4 Observations & Analysis

Which guard fires is decided by order: the assertion evaluates first, so Run 3 never reaches the b-check. The caught message is the program's diagnosis verbatim — "DivExp refuses b = 0" versus "assertion failed: a must be positive" — and the two names, `ZeroDivisionError` and `AssertionError`, are what a viva asks you to distinguish. No traceback reaches the screen; the except arms convert both into printed, gradeable lines.

7.5 Result

`DivExp` enforces its contract ($a > 0$) by assertion and refuses zero divisors by a raised, caught exception; all three paths — result, caught `ZeroDivisionError`, caught `AssertionError` — were executed and captured. CO1, CO5.

7.6 Viva Voce Questions

1. Why raise instead of return `None` for $b = 0$ in this design?
2. Assertion vs exception — which one is for programmer contracts and which for caller input? Justify from the transcript.
3. What line ends the program if the except clause were missing? Sketch the traceback.
4. Where does the assert message go in the output, and which exception type prefixes it?
5. Would `assert a > 0` still fire if python were run with `-O`? What does that imply for production code?

Model answers to the most-asked questions

Q1. Why raise instead of return `None` for $b = 0$ in this design?

Ans. `None` is a value the caller can forget to check — the division would keep flowing with a poison payload. Raising makes ignoring the error impossible and names it; the caller above prints a diagnosis instead of garbage.

Q5. Would `assert a > 0` still fire if python were run with `-O`? What does that imply for production code?

Ans. No — with `-O` assertions are skipped (optimised out). Production checks like these must be explicit `if/raise`; `assert` is for development-time postulates, which is precisely why the question pairs both mechanisms in one function.

7.7 Exercises & Extensions

1. Add `assert b != 0` instead of the raise and show the different transcript line.
2. Wrap the caller in a while-loop that re-asks after a failure; define the exit condition.

3. Return (quotient, remainder) as a tuple when both are valid; update the except arms.
4. Convert a/b to Fraction from the fractions module and print the exact rational for $12/4$ and $1/3$.

Experiment 8 — Complex Numbers as Objects — Summing N with a Pure Adder

The OOP capstone of the fixed set: a class with state and a printable face, a function that returns a new object (pure style, by the question's own words), and a fold that adds N by applying the pair rule N–1 times.

AIM	Define a Complex class; a function that takes two Complex objects and returns a new Complex with their sum; read N ($N \geq 2$) complex numbers and compute their addition.
OBJECTIVE	class/instance discipline, <code>__str__</code> for display, pure function returning new objects, fold pattern, input loop with validation.
ENVIRONMENT & MATERIALS	Python 3.13 on the laboratory Linux machines — identical behaviour in IDLE, VS Code or any CPython 3.8+ on Windows. Only the standard library (plus NumPy in Module 3 notes). Every transcript below was captured from a real pseudo-terminal run of the source printed in the same chapter.
COURSE OUT-COMES	CO5 (see the CO map in the front matter)

AS PRESCRIBED (VERBATIM)

8. Define a function that takes TWO objects representing complex numbers and returns a new complex number with the sum of two complex numbers. Define a suitable class 'Complex' to represent the complex number. Develop a program to read N ($N \geq 2$) complex numbers and to compute the addition of N complex numbers.

8.1 Approach

- `Complex(real, imag)` with an initialiser defaulting to 0 — one class, four lines, full state.
- `add_complex(x, y)` returns **`Complex(x.real+y.real, x.imag+y.imag)`** — a new object, neither argument mutated: the question says “returns a new complex number” and the program obeys it literally.
- Display via `__str__`: sign chosen from the imaginary part, absolute value printed, so `3–4i` renders correctly — the format work is one method, once, not six inline prints.
- The sum of N: start with `numbers[0]`, fold over the rest with the pair function; $N < 2$ exits with a message.

8.2 Program Code

The programs below are the ones executed in §8.3: listing and transcript come from the same files.

lab08.py · complete source (30 lines, max width 72 cols)

```
# Lab 8 -- Complex class; sum N (N >= 2) complex numbers via a function
# Run   : python3 lab08.py
class Complex:
    def __init__(self, real=0.0, imag=0.0):
        self.real = real
        self.imag = imag
    def __str__(self):
        sign = "+" if self.imag >= 0 else "-"
        return f"{self.real:g} {sign} {abs(self.imag):g}i"

def add_complex(x, y):
    """Return a NEW Complex holding the sum of x and y."""
    return Complex(x.real + y.real, x.imag + y.imag)

n = int(input("How many complex numbers (N >= 2)? "))
if n < 2:
    print("N must be at least 2.")
    raise SystemExit

numbers = []
```

```

for i in range(n):
    re = float(input(f"number {i + 1} real part: "))
    im = float(input(f"number {i + 1} imag part: "))
    numbers.append(Complex(re, im))

print("\nReading :", " ".join(str(z) for z in numbers))
total = numbers[0]
for z in numbers[1:]:
    total = add_complex(total, z)      # fold: sum of N via pairwise sum
print("Sum      :", total)

```

8.3 Execution & Transcripts

Invocation, typed exactly as shown:

```
python3 lab08.py
```

Captured terminal sessions follow — input appears on the same line as its prompt, echoed by the terminal, exactly as at the bench. Guard runs (bad input) are included on purpose: they are part of the evidence.

Run 1 — three numbers with mixed signs

```

How many complex numbers (N >= 2)? 3
number 1 real part: 1
number 1 imag part: 2
number 2 real part: 3
number 2 imag part: -4
number 3 real part: 0.5
number 3 imag part: 1.5

Reading : 1 + 2i  3 - 4i  0.5 + 1.5i
Sum      : 4.5 - 0.5i

```

Run 2 — conjugate pair cancels to 0 + 0i

```

How many complex numbers (N >= 2)? 2
number 1 real part: 0
number 1 imag part: -1
number 2 real part: 0
number 2 imag part: 1

Reading : 0 - 1i  0 + 1i
Sum      : 0 + 0i

```

8.4 Observations & Analysis

$(1+2i) + (3-4i) + (0.5+1.5i) = 4.5-0.5i$, and the conjugates $(0-1i)+(0+1i) = 0+0i$ — the zero imaginary part keeps its sign word from `__str__` (“+ 0i”), visible evidence that display and value are separate concerns. The fold touches no input object; a second print of the Reading line after the sum still shows the originals (try it — the exercise).

8.5 Result

A Complex class with proper initialisation and string conversion, a pure pairwise adder, and an N-way fold were implemented and verified, including the conjugate-cancellation edge and the $N \geq 2$ guard. CO5.

8.6 Viva Voce Questions

1. Give the two evidence lines that show `add_complex` is pure, not a modifier.
2. Why is `__str__` better here than a helper `display(z)`? One sentence for the caller side, one for the `print()`.
3. The question says $N \geq 2$ — where is that enforced and what is the failure output?
4. How does the program decide the sign in '3 - 4i'? Name the exact expression.

5. Which line makes the whole N-way sum; rewrite that line as a loop over pairs of your choice.

Model answers to the most-asked questions

Q1. Give the two evidence lines that show `add_complex` is pure, not a modifier.

Ans. The Reading line printed after the sum shows the originals unchanged, and `add_complex`'s return statement constructs `Complex(...)` instead of touching `x` or `y` — one is transcript evidence, one is code evidence.

Q5. Which line makes the whole N-way sum; rewrite that line as a loop over pairs of your choice.

Ans. `total = numbers[0]; for z in numbers[1:]: total = add_complex(total, z)` — that is the fold. An equivalent walk: pairwise reduce until one element remains; both are $N-1$ applications of the same binary rule, which is what the question's wording implies.

8.7 Exercises & Extensions

1. Add `__add__` to the class and `print(sum(numbers, Complex()))` — show the transcript changes only in verbosity.
2. Add multiply with $(a+bi)(c+di) = (ac-bd) + (ad+bc)i$ and verify $(1+1i)^2 = 0 + 2i$.
3. Store the N numbers as a list of tuples instead of objects; print the same sum; compare the two `__str__` burdens.
4. Read inputs in the single form `'3,-4'` per line (one `input()` call) — split + comprehension, then update the guard messages.

Experiment 9 — Text Analysis Tool (Open-Ended)

The first open-ended tool, built entirely from fixed-set parts: Module 2's string cleaning, Module 3's dictionary counting, Module 5's guard-before-work habit. The choices — what counts as a word, what counts as a sentence — are written down, because in this half of the lab the defense is the grade.

AIM	Build a tool that analyses a paragraph: frequency of each word, longest word, number of sentences, etc.
OBJECTIVE	design choices stated and honoured, token cleaning, counting and ranking reuse, a star-bar visual that costs one line, and an empty-input guard that never crashes.
ENVIRONMENT & MATERIALS	Python 3.13 on the laboratory Linux machines — identical behaviour in IDLE, VS Code or any CPython 3.8+ on Windows. Only the standard library (plus NumPy in Module 3 notes). Every transcript below was captured from a real pseudo-terminal run of the source printed in the same chapter.
COURSE OUT-COMES	CO2 + CO4 + CO5 (see the CO map in the front matter)

AS PRESCRIBED (VERBATIM)

9. Text Analysis Tool: Build a tool that analyses a paragraph: frequency of each word, longest word, number of sentences, etc.

9.1 Approach

- **Stated choices:** a word = whitespace token stripped of punctuation, lower-cased; a sentence = count of . ! ? (floored at 1 for a non-empty paragraph); "etc." = distinct count and total characters, free once words are a list.
- Longest via `max(words, key=len)` — one built-in where a hand loop would take four lines; ties resolve to the first occurrence, said in the record.
- Frequency ranking reuses the Lab-4 sort (`-count, word`) for deterministic order, and a `'**c` bar makes the histogram readable at a glance.
- Empty paragraph exits with a message — the tool never raises on nothing to do.

9.2 Program Code

The programs below are the ones executed in §9.3: listing and transcript come from the same files.

lab09.py · complete source (26 lines, max width 78 cols)

```
# Lab 9 (open-ended) -- text analysis: word frequency, longest word, sentences
# Run   : python3 lab09.py
import string

para = input("Paste the paragraph (one line): ").strip()
if not para:
    print("Empty paragraph -- nothing to analyse.")
    raise SystemExit

words = [w.strip(string.punctuation).lower() for w in para.split()]
words = [w for w in words if w]

freq = {}
for w in words:
    freq[w] = freq.get(w, 0) + 1    # count with a dictionary

sentences = max(sum(para.count(s) for s in ".!?"), 1)
longest = max(words, key=len)     # one built-in, no manual loop

print("\n===== TEXT ANALYSIS =====")
print(f"Words: {len(words)} | distinct: {len(freq)} | "
      f"characters: {len(para)} | sentences: {sentences}")
print(f"Longest word: '{longest}' ({len(longest)} chars)")
```

```
print("\nWord frequency (most -> least):")
for w, c in sorted(freq.items(), key=lambda kv: (-kv[1], kv[0])):
    print(f" {w:<12}{c:>2d} {'*' * c}")
```

9.3 Execution & Transcripts

Invocation, typed exactly as shown:

```
python3 lab09.py
```

Captured terminal sessions follow — input appears on the same line as its prompt, echoed by the terminal, exactly as at the bench. Guard runs (bad input) are included on purpose: they are part of the evidence.

Run 1 — a two-sentence paragraph with repeats

Paste the paragraph (one line): Python is fun and Python helps; fun times! Python rocks.

```
==== TEXT ANALYSIS ====
Words: 10 | distinct: 7 | characters: 56 | sentences: 2
Longest word: 'python' (6 chars)
```

```
Word frequency (most -> least):
python      3  ***
fun         2  **
and         1  *
helps       1  *
is          1  *
rocks       1  *
times       1  *
```

Run 2 — the empty guard

Paste the paragraph (one line):
Empty paragraph -- nothing to analyse.

9.4 Observations & Analysis

On “Python is fun and Python helps; fun times! Python rocks.”: 10 words, 7 distinct, 56 characters, 2 sentences (the ! and . each fire; the ; does not — a stated convention). The bar chart prints the 3-star python row above the 2-star fun row — the same –count tie-break as Lab 4, and the tool agrees with the hand count of every value in the summary block.

9.5 Result

A paragraph analysis tool was designed, implemented and tested; the word-frequency, longest-word and sentence-count requirements are met with explicit conventions, deterministic ranking and a graceful empty-input guard. CO2, CO4, CO5 in the open-ended rubric’s terms.

9.6 Viva Voce Questions

1. Why is stating “a word = token stripped of punctuation” worth marks even though it costs three lines?
2. Give the one-line change that would treat ';' as a sentence terminator too.
3. `max(words, key=len)` on two 6-letter words — which wins, and why is that acceptable?
4. Which single structure carries the whole frequency feature; name the module-3 idiom behind it.
5. If the paragraph came from a file instead of `input()`, which Lab’s two lines transfer verbatim?

Model answers to the most-asked questions

Q1. Why is stating “a word = token stripped of punctuation” worth marks even though it costs three lines?

Ans. Because an open-ended tool is graded on design, not just output: the convention decides every

number in the report, so writing it down turns 'our count says 10' into 'our count says 10 — by definition' and closes the viva question before it is asked.

Q5. If the paragraph came from a file instead of `input()`, which Lab's two lines transfer verbatim?

Ans. Lab 6's with `open(...) + readlines()` (or Lab 4's `read()` and one `split`) — the input layer is a swap, every analysis line is untouched — which is what building a tool, rather than a snippet, means.

9.7 Exercises & Extensions

1. Add average word length and a top-3 character histogram; keep all counts hand-checkable on the run's input.
2. Read the paragraph from a file chosen by `argv` (merge Lab 4's input style); rerun both inputs.
3. Add a stopwords list and an option to ignore them — one-line filter; report which counts move.
4. Extend the bar chart to a fixed 12-column width with scaling; state the rounding rule in a comment.

Experiment 10 — CSV Data Summary Generator (Open-Ended)

Rows to dictionaries, dictionaries to column stats, and a query loop that answers max, min and average for any numeric column — with the tool refusing, by name, what it cannot compute.

AIM	Read a CSV file (like COVID data or weather stats), convert to dictionary form, and allow the user to run summary queries: max, min, average by column.
OBJECTIVE	zip(header, row) → dict per row, numeric-column detection by float() with a caught ValueError, a menu loop with 'done', a query grammar the user cannot break.
ENVIRONMENT & MATERIALS	Python 3.13 on the laboratory Linux machines — identical behaviour in IDLE, VS Code or any CPython 3.8+ on Windows. Only the standard library (plus NumPy in Module 3 notes). Every transcript below was captured from a real pseudo-terminal run of the source printed in the same chapter.
COURSE OUT-COMES	CO2 + CO3 + CO4 (see the CO map in the front matter)

AS PRESCRIBED (VERBATIM)

10. Develop Data Summary Generator: Read a CSV file (like COVID data or weather stats), convert to dictionary form, and allow the user to run summary queries: max, min, average by column.

10.1 Approach

- **Dictionary form:** first line is the header; each data row becomes `dict(zip(header, cols))` — a list of records, one dict per line, as prescribed.
- **Column stats:** a column is numeric iff `float()` succeeds on every cell — the try/except does detection and cleaning in one pass, building a `numcols` dict of ready lists.
- Query grammar “op column” with `op ∈ {max, min, avg}`; every refusal prints the valid forms, so the loop is user-proof, and `done` exits.
- Weather CSV (city + three numeric columns) chosen as the data — small enough to verify by hand: max maxtemp 36.9 (Delhi), avg rain_mm 18.30, min mintemp 18.2.

10.2 Program Code

The programs below are the ones executed in §10.3: listing and transcript come from the same files.

lab10.py · complete source (39 lines, max width 91 cols)

```
# Lab 10 (open-ended) -- CSV -> list of dicts -> max/min/avg queries by column
# Run : python3 lab10.py
name = input("CSV file: ").strip()
try:
    with open(name) as f:
        rows = [ln.strip() for ln in f if ln.strip()]
except FileNotFoundError:
    print(f"'{name}' not found.")
    raise SystemExit

header = rows[0].split(",")
data = [dict(zip(header, r.split(","))) for r in rows[1:]]

numcols = {}
for col in header[1:]:
    try:
        numcols[col] = [float(d[col]) for d in data]
    except ValueError:
        pass
    # non-numeric column: no max/min/avg

print(f"Loaded {len(data)} rows | columns: {header}")
print(f"Numeric columns: {list(numcols)}")
```

```

while True:
    q = input("Query (max|min|avg <column>, 'done' to quit): ").split()
    if not q or q[0].lower() == "done":
        break
    op, *rest = q
    if len(rest) != 1 or rest[0] not in numcols or op.lower() not in ("max", "min", "avg"):
        print("    use: max|min|avg followed by a numeric column")
        continue
    col, vals = rest[0], numcols[rest[0]]
    if op.lower() == "avg":
        print(f"    average {col} = {sum(vals) / len(vals):.2f}")
    elif op.lower() == "max":
        print(f"    max {col} = {max(vals):g}")
    else:
        print(f"    min {col} = {min(vals):g}")
print("Bye.")

```

10.3 Execution & Transcripts

Invocation, typed exactly as shown:

```
python3 lab10.py
```

Captured terminal sessions follow — input appears on the same line as its prompt, echoed by the terminal, exactly as at the bench. Guard runs (bad input) are included on purpose: they are part of the evidence.

Run 1 — weather.csv, three queries then done

```

CSV file: ../data/weather.csv
Loaded 4 rows | columns: ['city', 'maxtemp', 'mintemp', 'rain_mm']
Numeric columns: ['maxtemp', 'mintemp', 'rain_mm']
Query (max|min|avg <column>, 'done' to quit): max maxtemp
    max maxtemp = 36.9
Query (max|min|avg <column>, 'done' to quit): avg rain_mm
    average rain_mm = 18.30
Query (max|min|avg <column>, 'done' to quit): min mintemp
    min mintemp = 18.2
Query (max|min|avg <column>, 'done' to quit): done
Bye.

```

Run 2 — missing file refused by name

```

CSV file: nope.csv
'nope.csv' not found.

```

10.4 Observations & Analysis

Loaded 4 rows; city correctly excluded from numeric columns by the float() test (its ValueError caught silently); each query line repeats its prompt, so the transcript reads as a conversation with the answer beside every ask. The missing-file run is the guard that replaces a traceback with 'nope.csv' not found.

10.5 Result

The generator converts CSV rows to dictionaries, identifies numeric columns automatically, and answers max/min/avg queries interactively with validated grammar and graceful file-error handling; hand-verified against the weather data. CO2, CO3, CO4.

10.6 Viva Voce Questions

1. Where exactly is the “dictionary form” created, and what does zip do if a row is short?
2. Which two lines make a column numeric or disqualify it? Name the exception caught.
3. avg rain_mm = 18.30 — reproduce that number with a hand sum of the four values.
4. Why print the prompt with the user’s query inside the transcript (pty vs pipe)?
5. State one design choice you would defend differently in the viva, and its cost.

Model answers to the most-asked questions

Q1. Where exactly is the “dictionary form” created, and what does zip do if a row is short?

Ans. `data = [dict(zip(header, r.split(","))) for r in rows[1:]]` — one comprehension. If a row is short, zip stops at the shorter sequence: that record simply lacks the trailing keys, which the numeric detector tolerates (KeyError is not even reached because `float()` never sees it) — a design choice with an explicit limit, said out loud.

Q4. Why print the prompt with the user’s query inside the transcript (pty vs pipe)?

Ans. Because the pty echo puts the typed query on the prompt line — the transcript then reads prompt → answer, the way the student actually sat at the bench; a piped run would show all inputs first and lose the conversation shape the record shows.

10.7 Exercises & Extensions

1. Add a 'sum ' query and a histogram bar of one chosen column; reuse Lab 9's star line.
2. Support quoted CSV fields by importing the csv module (reader/fDictReader) and re-running on the same file.
3. Write the chosen summary back to summary.txt with Lab 6's write discipline.
4. Let the user type 'filter city=Mangaluru' to restrict rows, then re-answer queries on the subset.

Experiment 11 — Student Grade Tracker (Open-Ended)

A list of dictionaries as the record store, one grading rule applied three times, and a report that answers the three questions every tracker is asked: how did each student do, who tops, who needs help.

AIM	Accept multiple students' names and marks; store them in a list of tuples or dictionaries; display summary reports (average, topper, etc.).
OBJECTIVE	sentinel-terminated input loop, per-student dict with computed fields, the report as aligned table, max with a key lambda for the topper, a list comprehension for the support list.
ENVIRONMENT & MATERIALS	Python 3.13 on the laboratory Linux machines — identical behaviour in IDLE, VS Code or any CPython 3.8+ on Windows. Only the standard library (plus NumPy in Module 3 notes). Every transcript below was captured from a real pseudo-terminal run of the source printed in the same chapter.
COURSE OUT-COMES	CO2 + CO5 (see the CO map in the front matter)

AS PRESCRIBED (VERBATIM)

11. Develop Student Grade Tracker: Accept multiple students' names and marks. Store them in a list of tuples or dictionaries. Display summary reports (average, topper, etc.).

11.1 Approach

- **Store:** one dict per student — name, marks (any count), avg, grade — appended to a list; dict chosen over tuple because the report reads by field name, which a viva can follow line by line.
- Input is free-form "Name,m1,m2,...", loop ends on a blank line; grades from one chained conditional expression (≥ 85 A, ≥ 70 B, ≥ 55 C, ≥ 40 D, else E) — the same table written once.
- **Reports:** the per-student table; topper via `max(students, key=lambda s: s["avg"])`; class average via a comprehension; support list via a filter comprehension with an "none" else.
- Empty input (blank first line) prints a clean refusal, not a crash on `max([])`.

11.2 Program Code

The programs below are the ones executed in §11.3: listing and transcript come from the same files.

lab11.py · complete source (34 lines, max width 79 cols)

```
# Lab 11 (open-ended) -- student grade tracker on a list of dicts
# Run   : python3 lab11.py
students = []
while True:
    line = input("Name,mark1,mark2,mark3 (blank line to finish): ").strip()
    if not line:
        break
    parts = [p.strip() for p in line.split(",")]
    if len(parts) < 2:
        print("    need at least a name and one mark -- skipped")
        continue
    name = parts[0]
    marks = [float(m) for m in parts[1:]]
    avg = sum(marks) / len(marks)
    grade = ("A" if avg >= 85 else "B" if avg >= 70 else
            "C" if avg >= 55 else "D" if avg >= 40 else "E")
    students.append({"name": name, "marks": marks, "avg": avg, "grade": grade})

if not students:
    print("No students entered -- nothing to report.")
    raise SystemExit

print("\n===== GRADE TRACKER REPORT =====")
print(f"{'STUDENT':<10}{'MARKS':<18}{'AVG':>6}{'GRADE':>6}")
for s in students:
```

```
ms = " ".join(f"{m:g}" for m in s["marks"])
print(f"{s['name']:<10}{ms:<18}{s['avg']:>6.2f}{s['grade']:>6}")

top = max(students, key=lambda s: s["avg"])
class_avg = sum(s["avg"] for s in students) / len(students)
print(f"\nStudents: {len(students)} | Topper: {top['name']} ({top['avg']:.2f})"
      f" | Class average: {class_avg:.2f}")
weak = [s["name"] for s in students if s["grade"] == "E"]
print("Needs support (grade E):", ", ".join(weak) if weak else "none")
```

11.3 Execution & Transcripts

Invocation, typed exactly as shown:

```
python3 lab11.py
```

Captured terminal sessions follow — input appears on the same line as its prompt, echoed by the terminal, exactly as at the bench. Guard runs (bad input) are included on purpose: they are part of the evidence.

Run 1 — three students, every grade band

```
Name,mark1,mark2,mark3 (blank line to finish): Asha,88,92,79
Name,mark1,mark2,mark3 (blank line to finish): Deepak,54,61,49
Name,mark1,mark2,mark3 (blank line to finish): Kiran,72,68,80
Name,mark1,mark2,mark3 (blank line to finish):
```

```
===== GRADE TRACKER REPORT =====
```

STUDENT	MARKS	AVG	GRADE
Asha	88 92 79	86.33	A
Deepak	54 61 49	54.67	D
Kiran	72 68 80	73.33	B

```
Students: 3 | Topper: Asha (86.33) | Class average: 71.44
Needs support (grade E): none
```

Run 2 — the empty roster guard

```
Name,mark1,mark2,mark3 (blank line to finish):
No students entered -- nothing to report.
```

11.4 Observations & Analysis

Asha 86.33 A, Kiran 73.33 B, Deepak 54.67 D; class average 71.44; no E — the support line prints “none” because the comprehension came back empty and the program says so rather than printing a blank. All numbers verify by hand: $88+92+79 = 259$, $259/3 = 86.33(3)$.

11.5 Result

The tracker accepts variable-length mark sets, stores typed records in a list of dictionaries, and produces the per-student table, topper, class average and support list — with the empty-roster guard verified. CO2, CO5.

11.6 Viva Voce Questions

1. Why dictionaries over tuples for a store with four fields?
2. Which line decides the topper and what does its key argument do?
3. Add a student with four subjects — which code lines change? Answer: name the number, it is zero.
4. State the grade boundary behaviour at exactly 70: inclusive or not, and where the answer lives.
5. Which single structure makes the report table print aligned — name the format feature.

Model answers to the most-asked questions

Q1. Why dictionaries over tuples for a store with four fields?

Ans. Field names in the report come free: `s["avg"]` reads the way the question words it; a tuple would be

s[2] with a comment, and every extra column shifts every index. Both satisfy the syllabus — the dict just survives the viva better.

Q3. Add a student with four subjects — which code lines change? Answer: name the number, it is zero.

Ans. None of the report or input lines: marks is a list of whatever length was typed, and every statistic folds over it (sum, len). That is the payoff of storing computed-per-record instead of fixed columns.

11.7 Exercises & Extensions

1. Add per-subject class averages by zipping rows — report the table with one extra comprehension.
2. Persist: dump the roster to students.csv with Lab 10's write discipline and reload it back into dicts.
3. Add a 'top n' command to the input loop that prints the top n by average; reuse the max pattern as a sort.
4. Model ties: two students at the same average — make the topper line report both with one list comprehension.

Experiment 12 — Recursive Folder Listing (Open-Ended)

The last tool is the module-3 “Directories” line given real depth: `os.listdir` at every level, `isdir` to decide descend, indentation as the recursion’s shadow.

AIM	Display contents of a folder recursively (Directory) having sub-folders and files (name and type).
OBJECTIVE	recursive function with a level parameter, sorted listing for deterministic output, name/type extraction via <code>splitext</code> , the <code> </code> -indent as a printed call stack.
ENVIRONMENT & MATERIALS	Python 3.13 on the laboratory Linux machines — identical behaviour in IDLE, VS Code or any CPython 3.8+ on Windows. Only the standard library (plus NumPy in Module 3 notes). Every transcript below was captured from a real pseudo-terminal run of the source printed in the same chapter.
COURSE OUT-COMES	CO3 + CO4 (see the CO map in the front matter)

AS PRESCRIBED (VERBATIM)

12. Develop a program to display contents of a folder recursively (Directory) having sub-folders and files (name and type).

12.1 Approach

- **walk(root, level):** `listdir`, `sort`; for each entry print the marker line (DIR or FILE with extension and byte size); recurse into directories with `level+1` — one function, six lines, any depth.
- Type from `os.path.splitext` — “(no extension)” printed explicitly, because a tool should say what it knows and what it lacks.
- **Sorted** at every level so the transcript (and a grader’s diff) is stable; indentation uses the `|` glyph so the tree is readable against a printed page.
- Non-directory input refuses by name after `os.path.isdir` — ghost-folder run proves it.

12.2 Program Code

The programs below are the ones executed in §12.3: listing and transcript come from the same files.

lab12.py · complete source (24 lines, max width 75 cols)

```
# Lab 12 (open-ended) -- recursive folder listing with name and type
# Run   : python3 lab12.py
import os

def walk(root, level=0):
    """Print every entry under root, indenting one level per folder."""
    entries = sorted(os.listdir(root))          # sorted: stable, printable
    for item in entries:
        path = os.path.join(root, item)
        if os.path.isdir(path):
            print("|" * level + f"└─ [DIR ] {item}")
            walk(path, level + 1)                # recursion into the folder
        else:
            ext = os.path.splitext(item)[1] or "(no extension)"
            size = os.path.getsize(path)
            print("|" * level + f"└─ [FILE] {item}  ({ext}, {size} B)")

root = input("Folder to list: ").strip() or "."
if not os.path.isdir(root):
    print(f"'{root}' is not a directory.")
    raise SystemExit

print(root)
walk(root)
```

12.3 Execution & Transcripts

Invocation, typed exactly as shown:

```
python3 lab12.py
```

Captured terminal sessions follow — input appears on the same line as its prompt, echoed by the terminal, exactly as at the bench. Guard runs (bad input) are included on purpose: they are part of the evidence.

Run 1 — the two-level demo tree

```
Folder to list: ../data/demo
../data/demo
├─ [FILE] README ((no extension), 12 B)
├─ [DIR ] photos
│   ├── [FILE] ship.png (.png, 2 B)
│   └── [FILE] sunset.jpg (.jpg, 1 B)
├─ [DIR ] project
│   ├── [FILE] ToDo.md (.md, 13 B)
│   ├── [FILE] notes.txt (.txt, 25 B)
│   └── [DIR ] src
│       └── [FILE] main.py (.py, 25 B)
```

Run 2 — a folder that does not exist

```
Folder to list: ghost-folder
'ghost-folder' is not a directory.
```

12.4 Observations & Analysis

The captured tree shows both branches under demo/ (photos with two image files; project with docs/, notes.txt, ToDo.md, src/main.py) and main.py's exact byte count — every line carries name and type as required. The empty docs/ folder prints nothing inside it, which is a result: the recursion saw it, found no entries, returned. Depth is visible in the indent count, which is the call stack rendered as art.

12.5 Result

A recursive directory listing tool was implemented with os.walk-grade information in os.listdir depth, deterministic sorted output, explicit type reporting, and a verified invalid path guard. CO3, CO4.

12.6 Viva Voce Questions

1. Why sort the entries at every call rather than once at the top?
2. Name the parameter that replaces the stack you would otherwise hand-manage, and its starting value.
3. What line of output would the program print for an empty directory — defend it with the code path.
4. Where does splitext lose the trailing dot on '.hidden' files, and does that matter here?
5. Which single os module function could replace the whole recursion, and what would the record lose?

Model answers to the most-asked questions

Q1. Why sort the entries at every call rather than once at the top?

Ans. Because recursion yields each directory in isolation — a directory sorted at the top would only order level 1; sortedness must hold at every listdir call to make the whole tree deterministic.

Q5. Which single os module function could replace the whole recursion, and what would the record lose?

Ans. os.walk — but the syllabus line says recursively, and the record's credit sits in showing the recursion (level parameter, descent decision, base case); a one-line library call would hide exactly the concept being examined.

12.7 Exercises & Extensions

1. Add a running total of bytes at the end; one accumulator through the recursion as a second parameter.
2. Filter mode: a second prompt for an extension ('.py'), and list only matching files — still showing directory names as context.
3. Print depth before each line instead of indentation and compare the readability in the record.
4. Write the tree to tree.txt using Lab 6's write discipline, then read it back to confirm line count equals file+directory count.

P A R T

C

Appendices

The tools the record, the internal and the theory paper are built from: a one-look Python quick reference keyed to the modules, the IPCC record format with the 25+25 rubric arithmetic and qualification working, a consolidated viva bank across the five modules and twelve lab questions, and the prescribed references exactly as listed.

IN THIS PART

- Appendix A — Python Quick Reference: values, operators, the str/list/dict methods the syllabus names, files, modules, classes, exceptions.
- Appendix B — Record Format & Assessment Rubrics: what each entry contains, how the CIE Practical 15 + 10 is awarded, and the eligibility arithmetic worked in full.
- Appendix C — Viva Voce Bank: recurring questions across modules and labs, with model answers at the depth the internal assessment expects.
- Appendix D — References & Further Reading: the syllabus list verbatim and the tools used to produce these notes.

Appendix A — Python Quick Reference

A.1 Values, names, expressions

Concept	Python
Basic types	<code>int · float · str · bool · NoneType</code> ; <code>type(v)</code> reports them
Conversion	<code>int("42") · float("4.2") · str(42) · float("42")</code> — no <code>int("4.2")</code>
Operators by precedence	<code>()</code> → <code>**</code> (right→left) → <code>-x</code> → <code>*</code> / <code>//</code> / <code>%</code> → <code>+</code> / <code>-</code> → comparisons → <code>not</code> / <code>and</code> / <code>or</code>
Division pair	<code>17//5 == 3 · 17%5 == 2 · 17 == 17//5*5 + 17%5</code>
String ops	<code>"ab"+"c"="abc" · "ab"*2="abab" · in / not in · s[i] · s[a:b] · len(s)</code>
Input	<code>input("Prompt: ")</code> returns <code>str</code> — convert at the edge

A.2 Control flow

Construct	Shape and one-line meaning
if ladder	<code>if ... elif ... else:</code> — one body runs; order the tests narrow-first
for	<code>for i in range(a, b, s):</code> — <code>b</code> excluded; step optional
while	<code>while cond:</code> — body must eventually falsify <code>cond</code>
break / continue	leave innermost loop / skip to next iteration
Nested loops	one loop level per data level (tables, matrices)

A.3 Collections — what the syllabus names by method

Type	The named members	Record-worthy notes
<code>str</code>	<code>strip lstrip rstrip lower upper find split join startswith endswith replace count format</code>	immutable: every method returns a NEW string; <code>find</code> → <code>-1</code> when absent
<code>tuple</code>	<code>() · count index · unpack a, b = t · return tuples</code>	fixed grouping; swappable without temp; hashable → dict keys
<code>list</code>	<code>append extend insert remove pop index count sort reverse clear · slices · del</code>	mutable; <code>sort()</code> is a modifier returning <code>None</code> ; <code>sorted()</code> the pure twin
<code>dict</code>	<code>d[k] get keys values items pop update copy clear</code>	key→value, insertion-ordered since 3.7; counting idiom <code>d[k]=d.get(k,0)+1</code>

A.4 Aliasing, copying, purity

- Assignment binds a name to an object; two names may bind one object (`b = a`; `b` is `a`).
- Immutable (`int/float/str/tuple`): sharing is invisible. Mutable (`list/dict/instances`): every alias sees every change.
- Shallow: `b = a.copy()` / `a[:]` — new top level, shared nested. Deep: `copy.deepcopy(a)` — fully separate.
- Pure function: returns new data, arguments untouched. Modifier: changes a caller-visible object (and usually returns `None` — the `nums.sort()` trap).

A.5 Files, directories, modules

Task	Code
Read all	<code>with open(p) as f: text = f.read()</code>

Task	Code
Read lines → list	<code>lines = [ln.strip() for ln in f.readlines() if ln.strip()]</code>
Stream	<code>for ln in f:</code> — one line in memory at a time
Write / append (text)	<code>open(p, "w") / "a" + f.write(s + "\n")</code> — you add the newline
Binary	<code>open(p, "rb"/"wb")</code> → bytes
Directory	<code>os.listdir, os.path.join/isdirexists, os.makedirs, os.walk</code>
Web	<code>from urllib.request import urlopen; urlopen(url).read().decode()</code>
Imports	<code>import m · from m import f · from m import f as g; own module = any .py beside it</code>
Scope	LEGB: Local → Enclosing → Global → Built-in; <code>global x</code> to rebind

A.6 NumPy in six lines

```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]])
a.shape          # (2, 3)          a.dtype — int64 here
a[:, 1]          # column as VIEW: writes through it change a
a[a > 3]         # boolean mask → [4 5 6]
a + np.array([10, 20, 30]) # broadcasting: row added to every row
a.sum(axis=0)    # column sums; axis=1 rows
```

A.7 Classes and exceptions

```
class Point:
    def __init__(self, x, y):          # state at creation
        self.x, self.y = x, y
    def __str__(self):                 # print(p) reads well
        return f"Point({self.x}, {self.y})"
    def __add__(self, o):              # overloading: p + q
        return Point(self.x + o.x, self.y + o.y)

try:
    risky()
except ValueError as e:               # narrow first
    print(e)
else:
    ok()
finally:
    cleanup()

if bad:
    raise ValueError("what is wrong and why") # your own exception
```

TOP INTERPRETER MESSAGES OF THIS COURSE

- `SyntaxError: invalid syntax` — missing colon/bracket; look at the line above too.
- `TypeError: can only concatenate str (not "int") to str` — the `input()`-without-conversion error.
- `IndexError: list index out of range` — len bounds; remember 0-based.
- `KeyError: 'z'` — use `get(k, default)` for counting/optional lookups.
- `ZeroDivisionError` — guard before dividing (Lab 7 does).

- `AttributeError: 'NoneType' object has no attribute 'append'` — you stored the result of a modifier: `x = nums.sort()`.
- `FileNotFoundError` — path/working-directory: relative paths hang off the CWD, not the `.py` file.

Appendix B — Record Format & Assessment Rubrics

B.1 The record entry, field by field

Field	Content expectation
Title / syllabus line	Experiment number and the question text verbatim (the a/b pair shares one entry, both titles listed).
Aim & objective	One sentence each; objective names the constructs practised, not the topic.
Approach / algorithm	The chosen route in 3–6 steps or bullets — for open-ended questions this is where design choices are stated, which is the graded half.
Program code	Complete listing as executed, filename on top; every listing line must match the file on the student's machine.
Output (transcript)	Captured session with input beside each prompt; at least one guard run (bad input) included.
Observations & analysis	Why the numbers are right — a hand check of at least one value; the table of test cases.
Result	Formal sentence: what was developed, what was verified, which CO it evidences.
Viva trail	The question list answered in one line each — the internal assessment is run from this page.

B.2 CIE Practical — the 25 marks, as the circular splits them

Component	Marks	Typical rubric lines
Continuous assessment (rubrics)	15	Record completeness & submission cadence (usually every 2 experiments) ~6; program correctness at the bench ~5; output discipline & edge runs ~4 — college rubric sheet governs.
Lab internal assessment (rubrics)	10	Live execution of a chosen program ~4; viva on that program's constructs ~3; troubleshooting the tutor plants (bad input) ~3.

B.3 CIE Theory — the 25 marks

Two internal assessment tests on the module content; the book's per-chapter checkpoint questions are written in the shape those papers take (short-answer + write-a-program-with-output). Split of 25 across the two tests follows the college calendar.

B.4 Qualification arithmetic — worked

Gate	Rule (circular wording)	Worked
SEE eligibility — theory	$\geq 40\%$ of 25 = 10	9/25 fails; 10/25 passes the gate.
SEE eligibility — practical	$\geq 40\%$ of 25 = 10	Both CIE halves must clear their own 10.
SEE pass	$\geq 35\%$ of 50 = 18	The circular states 18 (35% of 50 = 17.5, rounded up).
Course completion	$\text{CIE} + \text{SEE} \geq 40/100$	Gate-by-gate a student at 10+10+18 = 38 still misses 40 — every component is needed, none is spare.

READ THE GATES AS A LADDER

Two independent 10-mark gates open the SEE door; 18/50 clears the paper; 40/100 closes the course. The arithmetic above is the entire rule set — keep it on the inside cover of the record.

B.5 Running a 24-hour practical week and the record cadence

- 24 lab hours $\approx$ 12 sessions: one fixed question per session (both a/b parts inside the hour where applicable), open-ended 9–12 as the last four sessions with a mid-session demo.
- Write the record the same evening the output was captured — transcripts copied from the machine, not retyped from memory; a re-run takes four minutes and keeps the evidence honest.
- Before submission: run each program once more from a clean folder — the 15-mark rubric is mostly “does it still work when it is watched”.

Appendix C — Viva Voce Bank

Consolidated across the five modules and twelve experiments; the recurring questions are grouped first because they are asked in nearly every internal. Model answers follow the question groups; the chapter model answers in Part A/B are the long-form versions.

C.1 Recurring — asked in almost every viva

1. What is the difference between a syntax error, a runtime error and a semantic error? Which produces a traceback?
2. Why does `input()` always return a string, and what does the program do about it?
3. Explain `//` and `%` with 17 and 5; give one use of each.
4. Mutable vs immutable — which of `str`, `list`, `tuple`, `dict` can change after creation, and why does it matter for aliasing?
5. Assignment vs equality: what is `x = x + 1` doing, and why is it not an equation?
6. What is the difference between `break` and `continue`?
7. Write on the spot: a function with two parameters and a return value (any topic).
8. Difference of `a = b` and `a = b.copy()` for lists — and for dicts.
9. Why is `def f(x, bag=[])` dangerous? Give the fix.
10. What does `__str__` change about `print(obj)`, and why do lab programs need it?
11. `assert` vs `raise` — which is the developer contract, which is the caller-facing refusal?
12. File modes `w` vs `a` vs `r+`; what does a missing `\n` in `write()` do to the output file?

C.2 By module

1. **M1:** the three-step Collatz rule; why a `while` is natural for it; order of operations in `2 ** 3 ** 2`; the keyword list; why `int("4.2")` fails.
2. **M2:** slice both ends of “computer”; `s[0]="C"` — outcome and why; find returning `-1`; tuple swap mechanism; why `sort()` returns `None`; list as parameter: pure or modifier?
3. **M3:** the counting idiom line-by-line; dict key legality; mask vs boolean index; broadcasting with `(3,)` onto `(2,3)`; `readlines` vs `for-loop`; what `open()` “w” does to existing content.
4. **M4:** three import forms and their call sites; LEGB with an example; module execution happens once; self — what is it; instances passed: what does the callee see?
5. **M5:** `is` vs `==` on copies; shallow vs deep with a nested list; overriding vs overloading; a try ladder: which except fires; what disappears when `-O` runs?

C.3 By lab question (one-line answers)

1. **Q1:** convert at the edge; guard division before it; fixed year keeps transcripts honest — not real ages.
2. **Q2:** tuple assignment swaps without temp; `pop` is by index and returns; `remove` is by value; clear empties but does not destroy the object.
3. **Q3:** population variance $\div n$; digits counted from the string form; `get(k,0)+1` never `KeyErrors`.
4. **Q4:** sort key `(-count, word)` = descending then alphabetical; tie-break is what makes output reproducible.
5. **Q5:** bubble on an index permutation; `strict <=` stability; swap counter = hand-traceable evidence.
6. **Q6:** `readlines` + `strip` + `sort` + explicit “`\n`” writes; the output filename must not equal the input.
7. **Q7:** `assert` checks the contract, `raise` answers the caller; order decides which message appears first.
8. **Q8:** `add` returns a new `Complex` = pure; `fold` = `N-1` pair calls; `__str__` owns display, data owns values.
9. **Q9:** conventions are features: saying “word = token stripped & lowered” prevents the first viva question.
10. **Q10:** `zip(header, row)` is the dictionary form; `float()` detection with a caught `ValueError` marks numeric columns.
11. **Q11:** list-of-dicts stores per-record; `key=max` over `avg` is the topper; empty roster must short-circuit before `max([])`.

12. **Q12:** `sorted()` at every level or nowhere; the level parameter is the visible call stack; `os.walk` exists but recursion is the lesson.

C.4 How an internal viva runs (and how to open it well)

One program is chosen cold; the student runs it, then the questions above turn on it. Open with the sentence: “This program reads the marks into a list, bubble-sorts the indices, and the swap count you see is what my hand trace predicted.” — data flow, key construct, evidence. The whole answer is then in the examiner’s hands, and the rubric line the marker is filling (“explains program”, “troubleshoots when probed”) has its tick.

Appendix D — References & Further Reading

D.1 Prescribed in the syllabus

1. Peter Wentworth, Jeffrey Elkner, Allen B. Downey and Chris Meyers — How to Think Like a Computer Scientist: Learning with Python 3. Green Tea Press, Wellesley, Massachusetts, 2020. <https://media.readthedocs.org/pdf/howtothink/latest/howtothink.pdf> [Textbook — the chapter numbers in every module header point here.]
2. Al Sweigart — Automate the Boring Stuff with Python, 2nd Edition: Practical Programming for Total Beginners. No Starch Press, 2022. CC-BY-NC-SA at <https://automatetheboringstuff.com/> [Reference — files, CSV, and the practical instincts of Labs 9–12.]
3. Kyla McMullen, Elizabeth Matthews and Elizabeth Parsons — Programming with Python. Cengage, 2023. [Reference — the academic second read of the same core.]

D.2 E-resources from the syllabus

- Python tutorial and references: <https://www.learnbyexample.org/python/>
- Learn Python interactively: <https://www.learnpython.org/>
- Step-through visualiser (use it for the Collatz and bubble traces): <https://pythontutor.com/visualize.html#mode=edit>

D.3 Tools used to produce these notes

- CPython 3.13.14 with NumPy 2.3.5 — every program in the book was executed on it.
- Transcripts captured through a pseudo-terminal (pty) harness so echoed input lands after its prompt exactly as on the bench; no output was retyped or edited.
- Flow-free design on purpose: Python’s control flow is read from the code itself, and the record format this course grades does not ask for flowcharts — the approach bullets replace them.

D.4 For the curious

The Python Language Reference and the Standard Library docs (docs.python.org) for every “why does this work” question this lab generates; the PEP index for the history of the features used here (f-strings PEP 498, print function PEP 3105); and the textbook’s appendix of glossary entries, which doubles as the viva’s dictionary.

About the Author

Dr. Lokesh M R is Professor in the Department of Information Science & Engineering at A J Institute of Engineering and Technology (AJIET), Mangaluru, and teaches first-year computing courses under the VTU 2022 scheme, including this integrated Python course. His teaching practice centres on documentation that can be defended orally — the reason every chapter in this book pairs a worked program with a captured transcript and a viva trail. He is also the author of the companion activity notes for C Programming Lab and Introduction to AI and Applications in the same series and format.

Corrections, suggestions and classroom reports are welcome at the department office. First Edition, September 2026 — programs verified against CPython 3.13 on the publication date.